

FoodOS

Documentación Técnica Completa

App unificada de gestión alimentaria · Nutrición · Finanzas personales

Versión	3.0 — Mayo 2026
Estado del proyecto	Diseño completo / Pre-desarrollo
Plataformas	iOS · Android · Web (PWA + SSR)
Stack principal	Next.js 14 · Expo SDK 51 · Supabase · Vercel
IA principal (gratis)	Gemini 1.5 Flash · 1.500 req/día · Imagen 3 (con Google AI Plus)
IA alternativa pago	OpenAI GPT-4o / GPT-4o-mini
BYOK	Gemini · OpenAI · Claude · Ollama — API key propia del usuario
Coste operativo MVP	€0 / mes hasta ~500 usuarios activos
APIs externas	Open Food Facts · Nordigen PSD2 · Cloudinary
Base de datos	Supabase PostgreSQL con Row Level Security
Módulos	Inventario · Recetas · Carrito · Feed · Finanzas · Nutrición
Animaciones	GSAP 3.12.5 · Anime.js 3.2.2 · Lottie (mascotas)
Design System	Organic Dark · DM Serif Display · Outfit · DM Mono
Mascotas	15 personajes seleccionables · sprites Lottie · 9 estados animación
Secciones	23 secciones · 98 páginas de contenido técnico
Tiempo estimado MVP	7–9 semanas desarrollando solo

Índice de contenidos

1 Visión General y Propuesta de Valor

- 1.1 Qué es FoodOS y por qué existe
- 1.2 Diferenciadores frente a apps existentes
- 1.3 Público objetivo y casos de uso
- 1.4 Principios de diseño y UX

2 Stack Tecnológico — 100% Gratuito

- 2.1 Resumen del stack elegido
- 2.2 Frontend Web — Next.js 14 en Vercel
- 2.3 App Móvil — Expo / React Native
- 2.4 Autenticación — Supabase Auth
- 2.5 Base de datos — Supabase PostgreSQL
- 2.6 Storage — Supabase Storage + Cloudinary
- 2.7 Hosting y serverless — Vercel
- 2.8 Notificaciones push — Expo + Firebase FCM
- 2.9 Tabla completa de herramientas y límites gratuitos

3 Arquitectura del Sistema

- 3.1 Diagrama de capas
- 3.2 Monorepo con Turborepo
- 3.3 Row Level Security en Supabase

4 Módulo 1 — Inventario y Almacenes

- 4.1 Almacenes predefinidos y personalizados
- 4.2 Tres vías de entrada: manual, barcode, foto IA
- 4.3 Caducidades inteligentes por ubicación
- 4.4 Almacén compartido y propietarios
- 4.5 Notificaciones y alertas

5 Módulo 2 — Recetas

- 5.1 Creación y edición de recetas
- 5.2 Semáforo de disponibilidad de ingredientes
- 5.3 Escalado por personas y por calorías
- 5.4 Coste por ración · Recomendación IA

6 Módulo 3 — Carrito de Compra

- 6.1 Listas múltiples por tienda
- 6.2 Ingredientes cruzados entre recetas
- 6.3 Integración automática con finanzas

7 Módulo 4 — Feed Social

- 7.1 Publicar recetas con vídeo
- 7.2 Interacciones, guardado y filtros
- 7.3 Almacenamiento de vídeos con Cloudinary

8 Módulo 5 — Finanzas Personales

- 8.1 Gestión de ingresos y fuentes
- 8.2 Categorías de gasto
- 8.3 Conexión bancaria PSD2 — Nordigen (gratis)
- 8.4 Importación CSV / PDF / foto ticket
- 8.5 Presupuesto · Proyección de ahorro · Modo ahorro máximo

9 Módulo 6 — Nutrición y Objetivos Corporales

- 9.1 Perfil físico del usuario
- 9.2 Cálculo de TMB y TDEE — Mifflin-St Jeor
- 9.3 Modos: pérdida, ganancia, recomposición, mantenimiento
- 9.4 Ciclado calórico para recomposición corporal
- 9.5–9.9 Macros diarios, seguimiento, ajuste de porciones, optimizador proteína/€

10 Integraciones Cruzadas entre Módulos

- 10.1–10.6 Inventario↔Nutrición, Carrito↔Finanzas, Banco↔Inventario, etc.

11 Inteligencia Artificial — Implementación Completa

- 11.1 Gemini 1.5 Flash — GRATIS (1.500 req/día)
- 11.2 OpenAI GPT-4o — DE PAGO
- 11.3 Comparativa y estrategia recomendada
- 11.4–11.5 Casos de uso y prompts detallados
- 11.6–11.7 Rate limiting y estrategia de escalado

12 Modelo de Datos Completo

- 12.1 SQL completo de todas las tablas
- 12.2–12.4 Relaciones, RLS e índices

13 APIs Externas — Documentación de Uso

- 13.1 Open Food Facts API
- 13.2 Nordigen / GoCardless PSD2
- 13.3 Cloudinary · Firebase FCM

14 Roadmap y Fases de Desarrollo

- 14.1 Fase 0 — Setup (semana 1)
- 14.2 Fase 1 — MVP Core (semanas 2–7)
- 14.3 Fase 2 — IA y Finanzas (semanas 8–13)
- 14.4–14.6 Fase 3 · Costes por escala · Modelo de negocio

15 Módulo 7 — Búsqueda y Generación de Recetas por Ingrediente

- 15.1–15.4 4 modos de búsqueda · chips · semáforo · ordenación
- 15.5–15.6 Generación con IA · prompt completo
- 15.7–15.12 Persistencia · integraciones · modelo de datos · estimación IA

16 Landing Page — Página de Presentación

- 16.1–16.3 Propósito · secciones · video scrubbing
- 16.4–16.5 Stack técnico · gestión de rutas tras login

17 GSAP — GreenSock Animation Platform

- 17.1–17.3 Por qué GSAP · plugins · 5 casos de uso en FoodOS

17.4–17.5 Instalación en Next.js · buenas prácticas

18 **Anime.js — JavaScript Animation Engine**

18.1–18.2 GSAP vs Anime.js · 6 casos de uso en FoodOS

18.3 Estrategia cross-platform: Anime.js (web) + Reanimated (móvil)

19 **21st.dev Magic MCP — Generación de UI por IA en el IDE**

19.1–19.3 Qué es · 3 herramientas · instalación

19.4–19.6 5 comandos /ui para FoodOS · flujo · limitaciones

20 **Resumen — Ecosistema de Librerías de FoodOS**

20.1–20.3 Librerías animación · herramientas dev · tabla de decisión

21 **Design System — Identidad Visual de FoodOS**

21.1–21.2 Concepto Organic Dark · paleta completa con swatches

21.3–21.4 Tipografía · espaciado y bordes

21.5–21.6 Animaciones · principios · el logo

22 **Prompts Base — Generación de Assets Visuales**

22.1 Style Anchor — bloque fijo para todos los prompts

22.2–22.6 Prompts: alimentos, UI mockups, fondos, vídeos, iconografía

22.7 Herramientas gratuitas: Imagen 3, Firefly, Leonardo, Veo 2

23 **Mascotas Personales — Los 15 Compañeros de FoodOS**

23.1 Concepto: mascota personal, no asistente de módulo

23.2 Estilo visual unificado: Cuphead + Animal Crossing

23.3 Master Style Anchor — prompt ancla visual de los 15

23.4 Fichas completas y prompts de los 15 personajes

23.5–23.6 9 estados de animación · flujo de creación de sprites

23.7–23.9 Herramientas gratuitas · consistencia · código completo

1 Visión General y Propuesta de Valor

1.1 Qué es FoodOS y por qué existe

FoodOS es una aplicación multiplataforma (iOS, Android y web) que unifica en una sola interfaz tres áreas de la vida cotidiana que hoy están completamente desconectadas: la gestión del inventario de alimentos del hogar, el seguimiento de objetivos nutricionales y la gestión de las finanzas personales relacionadas con la alimentación.

El problema central que resuelve FoodOS es la desconexión entre lo que el usuario tiene en la nevera, lo que necesita comer para alcanzar sus objetivos de salud, y cuánto dinero puede o quiere gastar en comida. Hoy, estas tres preguntas se responden con tres apps distintas que no se hablan entre sí — y FoodOS las fusiona.

INTEGRACIÓN EN ACCIÓN

Ejemplo real: el usuario tiene pechuga de pollo que caduca mañana, le faltan 50g de proteína para cerrar sus macros del día (objetivo: recomposición corporal) y le quedan €28 del presupuesto semanal de comida. FoodOS lo sabe todo al mismo tiempo y genera una recomendación: 'Cena esta noche: 210g de pechuga con arroz integral 90g. Cubre tus macros, usa lo que caduca y cuesta €2,80 — dentro del presupuesto.'

1.2 Diferenciadores frente a apps existentes

App existente	Qué hace bien	Qué le falta	FoodOS lo resuelve
Yuka / Open Food Facts	Escaneo y macros de productos	No gestiona inventario ni finanzas	✓
MyFitnessPal	Seguimiento de calorías y macros	No conecta con inventario real ni presupuesto	✗
Bring! / OurGroceries	Lista de compra compartida	Sin macros, sin finanzas, sin inventario	✓
Fintonic / Wallet	Gestión financiera con banco	Sin integración con alimentación	✓
TikTok / Instagram	Feed de recetas en vídeo	Sin inventario, sin macros, sin presupuesto	✗
Mealime / Whisk	Planificación de recetas	Sin inventario real-time ni finanzas	✓

1.3 Público objetivo y casos de uso

Segmento	Necesidad principal	Módulos clave de FoodOS
Jóvenes adultos (18–35) que viven solos	Reducir desperdicio, optimizar presupuesto y aprender a comer mejor	Inventario + Finanzas + Nutrición
Personas con objetivos corporales activos	Control preciso de macros, calorías y coste de la dieta	Nutrición + Recetas + Inventario
Hogares compartidos (pisos, parejas, familias)	Coordinación de compras, gastos comunes y control de alimentos compartidos	Almacén compartido + Carrito + Finanzas
Usuarios interesados en cocina y comunidad	Inspiración culinaria, compartir recetas y descubrir tendencias	Feed social + Recetas

Segmento	Necesidad principal	Módulos clave de FoodOS
Personas con presupuesto muy ajustado	Maximizar el valor nutricional por euro gastado	Finanzas + Nutrición + Proteína/€

1.4 Principios de diseño y filosofía UX

- **Todo conectado, cero silos.** Cada acción en un módulo actualiza automáticamente los demás. Añadir un producto al carrito actualiza el presupuesto. Cocinar una receta descuenta los macros del día.
- **Facilitar, no complicar.** Añadir un alimento escaneando su código de barras tarda menos de 3 segundos. Las recomendaciones de IA son sugerencias, siempre editables y descartables.
- **Gratis por defecto.** Todas las funcionalidades principales están disponibles sin pago. El plan premium (si existe) solo desbloquea funciones avanzadas, nunca limita las básicas.
- **Privacidad por diseño.** Los datos bancarios nunca se almacenan en servidores de FoodOS. Los datos de salud están protegidos con RLS de nivel de base de datos.
- **IA como copiloto silencioso.** La inteligencia artificial trabaja en segundo plano. El usuario nunca necesita interactuar explícitamente con la IA para beneficiarse de ella.
- **Diseño mobile-first, web-complete.** La app móvil es el canal principal, pero la versión web tiene paridad total de funciones y es completamente responsiva.

2

Stack Tecnológico — 100% Gratuito

2.1 Resumen del stack elegido

El stack de FoodOS ha sido seleccionado con un criterio principal: máximo free tier sin comprometer calidad de producción. Todas las herramientas listadas tienen planes gratuitos suficientes para llegar a los primeros 500 usuarios activos sin ningún coste mensual.

CRITERIO DE SELECCIÓN DEL STACK

REGLA DE ORO: Si una herramienta no tiene un plan gratuito generoso, no entra en el stack. El objetivo es que el proyecto pueda existir, crecer y validarse sin inversión económica inicial.

2.2 Frontend Web — Next.js 14 en Vercel

Next.js 14 con App Router es el framework elegido para la web. Permite combinar páginas estáticas (SSG) para las recetas públicas del feed (mejora el SEO de forma drástica), páginas renderizadas en servidor (SSR) para el dashboard personalizado, y rutas de API serverless para todo el backend — sin necesidad de un servidor dedicado.

- **App Router (Next.js 14):** layouts anidados, streaming SSR, Server Components para reducir JS enviado al cliente.
- **API Routes como backend:** cada endpoint vive en `/app/api/[ruta]/route.ts` y se despliega como función serverless en Vercel.
- **SEO automático:** las recetas públicas del feed se generan estáticamente y son indexables por Google.
- **PWA:** con next-pwa, la versión web puede instalarse en el móvil como app nativa sin pasar por las stores.
- **TypeScript estricto:** tipos compartidos entre web y móvil en el monorepo.

Vercel Hobby Plan — límites gratuitos:

Recurso	Límite gratuito	Suficiente para...
Deploys	Sin límite	Desarrollo continuo
Bandwidth	100 GB/mes	~50.000 visitas/mes con assets normales
Funciones serverless	100 GB-horas/mes	Millones de invocaciones pequeñas
Builds	6.000 min/mes	Decenas de deploys diarios
Edge Network	Global CDN incluido	Latencia baja en todo el mundo
Dominio custom	1 dominio gratis	foodos.vercel.app o dominio propio
SSL/HTTPS	Automático	Certificado Let's Encrypt gestionado

2.3 App Móvil — Expo / React Native

Expo es el framework elegido para la app móvil porque permite desarrollar iOS y Android con un único codebase en JavaScript/TypeScript, compartiendo casi toda la lógica con la versión web. Con Expo Go, el desarrollador puede probar la app en su propio dispositivo físico sin publicar en ninguna store.

- **Expo SDK 51:** acceso a cámara, GPS, barcode scanner, notificaciones push, biometría y más.
- **Expo Go:** durante desarrollo, la app se prueba sin compilar — escanea un QR y listo.
- **EAS Build:** compilación en la nube. Plan gratuito: 30 builds/mes (suficiente para desarrollo).
- **EAS Submit:** publicación directa a Google Play y App Store desde la CLI.
- **React Navigation:** navegación nativa con tabs, stacks y modales.
- **Expo Notifications:** push notifications con Firebase FCM, completamente gratuito.

ÚNICO COSTE INEVITABLE

COSTE DE LAS STORES: Google Play tiene un pago único de 25 USD. App Store de Apple requiere 99 USD/año. Estos son los únicos costes inevitables para publicar en las tiendas oficiales. Durante el desarrollo y las fases de prueba (Testflight en iOS, Play Internal Testing en Android) no hay coste adicional.

2.4 Autenticación — Supabase Auth + Firebase Auth

Supabase Auth es la opción principal para la autenticación. Firebase Auth se documenta como alternativa por su popularidad y su integración nativa con el ecosistema de Google, especialmente útil si se usa Firebase FCM para notificaciones.

Característica	Supabase Auth (principal)	Firebase Auth (alternativa)
Email/contraseña	✓ Gratis	✓ Gratis
Google OAuth	✓ Gratis	✓ Gratis
Apple OAuth	✓ Gratis	✓ Gratis
Magic Link (email)	✓ Gratis	✓ Gratis
SMS/OTP	De pago	✓ Gratis (10k/mes)
Usuarios gratuitos	50.000/mes	Ilimitado
Integración con DB	Nativa (PostgreSQL)	Requiere Firestore separado
RLS automático	✓ (con JWT claims)	Manual en reglas Firestore
Precio al escalar	€25/mes (Pro)	Según uso (Spark plan gratis)

Recomendación: usar Supabase Auth como sistema principal por su integración perfecta con la base de datos PostgreSQL y el RLS automático. Si se necesita SMS en el futuro, se puede añadir Firebase Auth solo para ese proveedor.

2.5 Base de datos — Supabase PostgreSQL

Supabase provee PostgreSQL gestionado con Row Level Security (RLS), subscripciones en tiempo real (WebSockets), funciones Edge y un cliente JavaScript (@supabase/supabase-js) que funciona tanto en Next.js como en Expo.

Recurso Supabase Free	Límite	¿Suficiente para MVP?
Base de datos PostgreSQL	500 MB almacenamiento	Sí (miles de usuarios y productos)
Usuarios activos/mes	50.000	Sí, hasta escala considerable
Storage de archivos	1 GB	Suficiente para fotos, no para vídeos

Recurso Supabase Free	Límite	¿Suficiente para MVP?
Ancho de banda storage	2 GB/mes	Suficiente para MVP
Edge Functions	500.000 invocaciones/mes	Más que suficiente
Realtime (websockets)	500 conexiones activas	Suficiente para MVP
Proyectos gratuitos	2 proyectos	Suficiente (1 dev + 1 prod)
Pausa por inactividad	Pausa tras 7 días sin uso	Solo afecta en desarrollo
Backups	Backup diario	Sí

2.6 Storage — Supabase + Cloudinary

Se usa una estrategia de storage dual: Supabase Storage para fotos de alimentos y thumbnails de recetas (archivos pequeños, acceso frecuente), y Cloudinary para los vídeos del feed social (transformaciones, streaming adaptativo, thumbnails automáticos).

- **Supabase Storage (gratis: 1 GB + 2 GB BW):** fotos de productos escaneados, avatares de usuarios, imágenes de recetas.
- **Cloudinary Free (25 GB storage + 25 GB BW/mes):** vídeos del feed social. Genera thumbnails automáticamente, comprime y adapta la calidad según la red.
- **Open Food Facts CDN:** las imágenes de productos de la API se sirven directamente desde sus servidores — sin coste ni almacenamiento propio.

2.7 Notificaciones Push — Expo + Firebase FCM

Firebase Cloud Messaging (FCM) es completamente gratuito y sin límite de mensajes. Se integra con Expo Notifications para enviar push tanto a iOS como a Android con una única llamada a la API.

```
// lib/notifications.ts – Enviar push desde servidor (Next.js API Route)

import { Expo, ExpoPushMessage } from 'expo-server-sdk';

const expo = new Expo();

export async function sendPushNotification(
  tokens: string[],
  title: string,
  body: string,
  data?: Record<string, string>
) {
  const messages: ExpoPushMessage[] = tokens
    .filter(token => Expo.isExpoPushToken(token))
    .map(token => ({ to: token, sound: 'default', title, body, data }));

  const chunks = expo.chunkPushNotifications(messages);
```

```
for (const chunk of chunks) {  
  await expo.sendPushNotificationsAsync(chunk);  
}  
}  
  
// Uso: caducidad próxima  
sendPushNotification(  
  [userPushToken],  
  '■ Caduca mañana',  
  'La leche entera caduca mañana. ¿La usas hoy?',  
  { type: 'expiry', itemId: 'abc123' }  
);
```

2.9 Tabla completa de stack con límites gratuitos

Herramienta	Función	Plan gratuito	Límite clave	Precio al escalar
Next.js 14	Web frontend + API	OSS / MIT	Sin límite (self-hosted posible)	€0 siempre
Expo SDK 51	App iOS/Android	OSS / MIT	Sin límite (Expo Go dev)	EAS: 50\$/mes
Supabase	DB + Auth + Storage + RT	Free tier	500MB DB · 50k users	€25/mes (Pro)
Vercel	Deploy + serverless	Hobby plan	100GB BW · 6k min builds	€20/mes (Pro)
Firebase Auth	Auth alternativa	Spark plan	Ilimitado en auth	Pay as you go
Firebase FCM	Push notifications	Spark plan	Sin límite de mensajes	€0 siempre
Cloudinary	Vídeos e imágenes	Free tier	25GB storage + 25GB BW	€0–89/mes
Open Food Facts	API de alimentos	Open source	Sin límite, sin auth	€0 siempre
Nordigen/GoCardless	PSD2 bancario	Free plan	50 usuarios banco	€0–50/mes
Gemini 1.5 Flash	IA principal (gratis)	Google AI Studio	1.500 req/día	€0.075/1M tokens
Turborepo	Monorepo	OSS / MIT	Sin límite	€0 siempre
TypeScript	Tipado estático	OSS / Apache	Sin límite	€0 siempre
TailwindCSS	Estilos web	OSS / MIT	Sin límite	€0 siempre
Zustand	Estado global	OSS / MIT	Sin límite	€0 siempre
React Query v5	Caché y sync datos	OSS / MIT	Sin límite	€0 siempre
Zod	Validación schemas	OSS / MIT	Sin límite	€0 siempre
GitHub	Control de versiones	Free plan	Repos privados ilimitados	€0/siempre

3 Arquitectura del Sistema

3.1 Diagrama de capas del sistema

CAPA	TECNOLOGÍA	DEPLOY	COMUNICACIÓN
Cliente Web	Next.js 14 (React)	Vercel Edge Network	HTTP/HTTPS · WebSockets
Cliente Móvil iOS	Expo / React Native	App Store (distribución)	HTTP/HTTPS · WebSockets
Cliente Móvil Android	Expo / React Native	Google Play (distribución)	HTTP/HTTPS · WebSockets
API / Backend	Next.js API Routes (serverless)	Vercel Functions	REST + JSON
Auth	Supabase Auth	Supabase Cloud	JWT tokens
Base de datos	PostgreSQL (Supabase)	Supabase Cloud	PostgREST + Realtime WS
Storage media	Supabase Storage + Cloudinary	CDN global	HTTPS signed URLs
IA	Gemini 1.5 Flash	Google AI Studio	REST + JSON
Push notifications	Firebase FCM + Expo	Firebase + Expo Push Svc	FCM protocol
API Alimentos	Open Food Facts	Open Food Facts CDN	REST + JSON
API Bancaria	Nordigen/GoCardless	Nordigen Cloud	OAuth2 + REST

3.2 Estructura del monorepo con Turborepo

El proyecto usa un monorepo gestionado con Turborepo. Esto permite compartir tipos TypeScript, componentes UI, hooks y lógica de negocio entre la web y la app móvil, evitando duplicación de código y manteniendo consistencia.

```
foodos/                                ← Raíz del monorepo
├── apps/
│   ├── web/                            ← Next.js 14 (web app + API routes)
│   └── app/
│       ├── (auth)/                     ← Login, registro, recuperar contraseña
│       ├── (dashboard)/                ← Pantallas principales (auth required)
│       ├── home/
│       ├── almacen/
│       ├── recetas/
│       ├── carrito/
│       ├── finanzas/
│       ├── nutricion/
│       ├── feed/
│       ├── api/                        ← API routes serverless
│       └── ai/                         ← Endpoints de IA (proxy a Gemini)
```

```

■ ■ ■ ■ ■ ■■■ bank/           ← Nordigen PSD2 endpoints
■ ■ ■ ■ ■ ■■■ inventory/
■ ■ ■ ■ ■ ■■■ nutrition/
■ ■ ■ ■ ■ ■■■ layout.tsx
■ ■ ■■■ package.json
■ ■■■ mobile/                 ← Expo app (iOS + Android)
■ ■■■ app/                    ← Expo Router (file-based routing)
■ ■ ■■■ (tabs)/
■ ■ ■ ■■■ index.tsx          ← Home
■ ■ ■ ■■■ almacen.tsx
■ ■ ■ ■■■ recetas.tsx
■ ■ ■ ■■■ carrito.tsx
■ ■ ■ ■■■ finanzas.tsx
■ ■ ■■■ modals/
■ ■■■ package.json

■■■ packages/
■ ■■■ ui/                     ← Componentes React compartidos
■ ■■■ types/                  ← Interfaces TypeScript (User, Recipe, Item...)
■ ■■■ utils/                  ← Funciones puras (cálculos TMB, macros, etc.)
■ ■■■ api-client/             ← Supabase client + queries reutilizables
■ ■■■ config/                 ← ESLint, TypeScript, Tailwind config
■■■ turbo.json
■■■ package.json
```

3.3 Row Level Security — seguridad a nivel de base de datos

Row Level Security (RLS) en PostgreSQL/Supabase permite definir qué filas puede ver o modificar cada usuario directamente en la base de datos, independientemente de cómo se llame la API. Es la capa de seguridad más robusta posible: aunque la lógica del servidor tenga un bug, la base de datos nunca devuelve datos que no corresponden al usuario autenticado.

Tabla	Política SELECT	Política INSERT/UPDATE/DELETE
inventory_items	almacen.members @> auth.uid()	item.owner_id = auth.uid() OR almacen.owner
almacenes	owner_id = auth.uid() OR member	owner_id = auth.uid()
gastos	user_id = auth.uid()	user_id = auth.uid()
food_log	user_id = auth.uid()	user_id = auth.uid()
user_profiles	user_id = auth.uid()	user_id = auth.uid()

Tabla	Política SELECT	Política INSERT/UPDATE/DELETE
recipes	is_public = true OR user_id = auth.uid()	user_id = auth.uid()
bank_connections	user_id = auth.uid()	user_id = auth.uid()
objetivos_ahorro	user_id = auth.uid()	user_id = auth.uid()

4 Módulo 1 — Inventario y Almacenes

4.1 Almacenes predefinidos y personalizados

El módulo de inventario es el núcleo de toda la aplicación. Su estado es consultado por prácticamente todos los demás módulos. FoodOS crea automáticamente tres almacenes predefinidos al registrar una cuenta nueva: Nevera, Congelador y Despensa. El usuario puede crear almacenes adicionales (ej: 'Bodega', 'Maletero', 'Oficina').

- **Nevera:** temperatura 2-8°C. Las caducidades de los productos se calculan para este rango.
- **Congelador:** temperatura -18°C. Los mismos productos duran mucho más que en nevera (ej: pechuga de pollo — nevera 7 días, congelador 90 días).
- **Despensa:** temperatura ambiente. Para conservas, secos, aceites, especias.
- **Almacenes custom:** nombre libre, sin caducidades automáticas predefinidas.

4.2 Tres vías de entrada de alimentos

Vía 1 — Búsqueda manual con autocompletado:

El usuario escribe el nombre del producto y la app consulta Open Food Facts API en tiempo real. Al seleccionar un producto del autocompletado, se rellenan automáticamente todos los campos: nombre, macros por 100g, y fecha de caducidad estimada según el almacén seleccionado.

Vía 2 — Escaneo de código de barras:

```
// hooks/useBarcodeScan.ts (Expo)

import { BarCodeScanner } from 'expo-barcode-scanner';

import { supabase } from '@lib/supabase';

export function useBarcodeScan(onResult: (product: FoodProduct) => void) {

  const handleScan = async ({ type, data }: BarCodeScannerResult) => {

    // 1. Buscar en Open Food Facts por EAN/UPC

    const res = await fetch(

      `https://world.openfoodfacts.org/api/v2/product/${data}.json`

    );

    const json = await res.json();

    if (json.status === 1) {

      const p = json.product;

      onResult({

        name: p.product_name_es || p.product_name,

        barcode: data,

        kcal: p.nutriments?.['energy-kcal_100g'],

        protein: p.nutriments?.proteins_100g,
```

```
        carbs: p.nutriments?.carbohydrates_100g,
        fat: p.nutriments?.fat_100g,
        imageUrl: p.image_front_url,
        brands: p.brands,
    });
} else {
    // Producto no encontrado: abrir formulario manual pre-rellenado con el barcode
    onResult({ barcode: data, name: '', kcal: null, protein: null, carbs: null, fat: null });
}
};

return { handleScan };
}
```

Vía 3 — Foto con detección IA (Gemini Vision):

El usuario hace una foto al producto (sin necesidad de barcode) y Gemini 1.5 Flash identifica el alimento, estima la cantidad visible y proporciona macros estimados. Antes de guardar, el usuario puede editar todos los campos.

```
// app/api/ai/detect-food/route.ts

export async function POST(req: Request) {
    const { imageBase64, almacenType } = await req.json();

    const prompt = `Analiza esta imagen de alimento. Responde ÚNICAMENTE en JSON válido:
{
    "name": "nombre del alimento en español",
    "quantity_estimate": número,
    "unit": "g|ml|ud|kg",
    "confidence": 0-1,
    "macros_per_100g": {
        "kcal": número,
        "protein": número,
        "carbs": número,
        "fat": número
    },
    "shelf_life_days": {
        "fridge": número,
        "freezer": número,
        "pantry": número
    }
}
```

```
    }

    }`;

    const response = await fetch(

      `https://generativelanguage.googleapis.com/v1beta/models/gemini-1.5-flash:generateContent?key=${process.env.GEMINI_API_KEY}`,

      {

        method: 'POST',

        headers: { 'Content-Type': 'application/json' },

        body: JSON.stringify({

          contents: [{

            parts: [

              { text: prompt },

              { inline_data: { mime_type: 'image/jpeg', data: imageBase64 } }

            ]

          }],

          generationConfig: { temperature: 0.1 }

        })

      }

    );

    const data = await response.json();

    const text = data.candidates[0].content.parts[0].text;

    // Limpiar posibles markdown fences antes de parsear

    const clean = text.replace(/```json|```/g, '').trim();

    return Response.json(JSON.parse(clean));

  }

}
```

4.3 Caducidades inteligentes por ubicación

Al añadir un alimento a un almacén, si el producto se reconoce en Open Food Facts o por IA, la fecha de caducidad se calcula automáticamente según el tipo de almacén. El usuario siempre puede editar esta fecha manualmente.

Alimento	Nevera (días)	Congelador (días)	Despensa (días)
Pechuga de pollo cruda	5–7	90	—
Leche entera abierta	5–7	30	—
Huevos (cáscara intacta)	21–28	365	21 (temp. ambiente)
Arroz (seco)	—	—	365–730

Alimento	Nevera (días)	Congelador (días)	Despensa (días)
Pasta seca	—	—	730
Yogur	14	60	—
Queso fresco	7	30	—
Frutas y verduras	3–7	90–180	1–3 (temperatura amb.)
Conservas (bote cerrado)	—	—	730–1460

4.5 Almacén compartido

Varios usuarios pueden compartir un mismo almacén. Cada alimento tiene un propietario (el usuario que lo añadió). Si otro miembro del almacén consume un alimento que no es suyo, el propietario recibe una notificación push, puede ver quién lo consumió y añadirlo al carrito con un solo toque.

- El propietario del almacén puede invitar a otros usuarios por email o enlace.
- Roles: propietario (puede eliminar miembros) y miembro (puede añadir/consumir).
- Cada ítem del inventario tiene un campo `owner_id` con el UUID del creador.
- Al marcar un ítem como consumido por otro usuario, se inserta un registro en `consumption_log` con el `user_id` del consumidor.
- La notificación se envía al propietario vía Firebase FCM con el nombre del consumidor y el producto.

5

Módulo 2 — Recetas

5.1 Creación y edición de recetas

- Título, descripción, tiempo de preparación, número de raciones.
- Lista de ingredientes con cantidad, unidad y macros (autocompletados desde Open Food Facts).
- Pasos de preparación con texto y fotos opcionales por paso.
- Imagen principal o vídeo corto para el feed social.
- Visibilidad: privada (solo yo), compartida (solo almacén) o pública (feed).
- Tags: alta proteína, vegano, sin gluten, rápida, económica, etc.

5.2 Semáforo de disponibilidad de ingredientes

Color	Significado	Condición técnica
■ Verde	Tienes suficiente	item en inventario AND cantidad >= requerida
■ Amarillo	Tienes pero poco	item en inventario AND cantidad < requerida AND cantidad > 0
■ Rojo	No tienes / no está	item NOT en inventario OR cantidad = 0

5.3 Escalado por personas y por calorías objetivo

Las recetas pueden escalarse de dos formas independientes o combinadas:

- **Por número de personas:** todos los ingredientes se multiplican proporcionalmente. Ej: receta para 2 → escalar a 4 personas duplica todas las cantidades.
- **Por calorías objetivo:** el usuario introduce las kcal que necesita de esta comida (ej: 'necesito 600 kcal en la cena') y la app calcula las porciones exactas de cada ingrediente.

```
// utils/recipe-scaler.ts

export function scaleByCalories(recipe: Recipe, targetKcal: number): ScaledRecipe {

  const ratio = targetKcal / recipe.kcalPerServing;

  return {

    ...recipe,
    servings: ratio,
    ingredients: recipe.ingredients.map(ing => ({
      ...ing,
      quantity: Math.round(ing.quantity * ratio * 10) / 10
    })),
    macros: {
      kcal: Math.round(recipe.macrosPerServing.kcal * ratio),
      protein: Math.round(recipe.macrosPerServing.protein * ratio * 10) / 10,
    }
  }
}
```

```
        carbs:    Math.round(recipe.macrosPerServing.carbs    * ratio * 10) / 10,  
        fat:      Math.round(recipe.macrosPerServing.fat      * ratio * 10) / 10,  
    }  
};  
}
```

5.5 Recomendación de recetas por IA

Al pulsar 'Recomiéndame algo', la IA recibe el inventario actual del usuario, su perfil nutricional, los macros pendientes del día y el presupuesto de comida restante. Devuelve 3–5 recetas personalizadas con la porción exacta recomendada.

El prompt completo de esta función se documenta en la sección §11 (IA).

6 Módulo 3 — Carrito de Compra

6.1 Listas múltiples por tienda

El carrito puede contener múltiples listas simultáneas, cada una asociada a una tienda diferente (Mercadona, Lidl, Frutería, Carnicería). Esto permite organizar la compra antes de salir de casa.

6.2 Ingredientes cruzados entre recetas

Si se añaden dos recetas al carrito y ambas necesitan 'arroz integral', FoodOS suma las cantidades en un único ítem en lugar de duplicarlas. Al marcar el ítem como comprado, se actualiza en todas las recetas que lo usan.

```
// utils/cart-merge.ts – Fusionar ingredientes de múltiples recetas

export function mergeCartIngredients(recipes: Recipe[]): CartItem[] {

  const map = new Map<string, CartItem>();

  for (const recipe of recipes) {

    for (const ing of recipe.ingredients) {

      const key = ing.name.toLowerCase().trim();

      if (map.has(key)) {

        const existing = map.get(key)!;

        existing.totalQuantity += ing.quantity;

        existing.recipeRefs.push(recipe.id);

      } else {

        map.set(key, {

          name: ing.name,

          unit: ing.unit,

          totalQuantity: ing.quantity,

          recipeRefs: [recipe.id],

          checked: false,

          estimatedPrice: null

        });

      }

    }

  }

  return Array.from(map.values());

}
```

6.3 Integración automática con finanzas al completar

Al pulsar 'Marcar compra como completada', FoodOS ejecuta automáticamente:

1. Crea una transacción en la tabla gastos con el importe total del carrito (manual o estimado por IA), categoría 'Comida', fecha de hoy y source = 'foodos_cart'.
2. Si el banco está conectado y hay un cargo de supermercado ese mismo día, el sistema intenta reconciliar automáticamente para evitar duplicados (compara comercio + importe $\pm$ 5%).
3. Actualiza el inventario añadiendo todos los productos comprados.
4. Limpia el carrito y archiva la lista para el historial.

7 Módulo 4 — Feed Social

7.1 Publicar recetas con vídeo

- Vídeo corto (máx. 60 segundos, formato vertical) o galería de fotos por pasos.
- Ingredientes con cantidades, macros calculados automáticamente.
- Tiempo de preparación, dificultad y tags (alta proteína, vegano, económica...).
- El vídeo se sube directamente a Cloudinary desde el cliente (upload signed), sin pasar por el servidor de FoodOS.
- Cloudinary genera automáticamente: thumbnail, versión comprimida, HLS para streaming adaptativo.

7.4 Almacenamiento de vídeos con Cloudinary (gratis)

Cloudinary es la solución elegida para los vídeos del feed porque su free tier incluye 25 GB de almacenamiento y 25 GB de bandwidth mensual — suficiente para un MVP con cientos de vídeos cortos. La subida se hace con signed uploads directamente desde el cliente móvil o web.

```
// app/api/cloudinary/sign/route.ts – Generar firma para upload seguro

import { v2 as cloudinary } from 'cloudinary';

cloudinary.config({
  cloud_name: process.env.CLOUDINARY_CLOUD_NAME,
  api_key: process.env.CLOUDINARY_API_KEY,
  api_secret: process.env.CLOUDINARY_API_SECRET,
});

export async function POST(req: Request) {
  const { folder, public_id } = await req.json();

  const timestamp = Math.round(Date.now() / 1000);
  const signature = cloudinary.utils.api_sign_request(
    { folder, public_id, timestamp, resource_type: 'video' },
    process.env.CLOUDINARY_API_SECRET!
  );

  return Response.json({ signature, timestamp,
    cloudName: process.env.CLOUDINARY_CLOUD_NAME,
    apiKey: process.env.CLOUDINARY_API_KEY });
}
```

8

Módulo 5 — Finanzas Personales

8.1 Gestión de ingresos y fuentes

El usuario puede registrar sus fuentes de ingreso con nombre, importe, frecuencia (mensual, quincenal, anual) y día del mes en que se cobra. FoodOS usa esto para calcular el balance mensual y las proyecciones de ahorro. Las fuentes de ingreso pueden ser múltiples (nómina principal, trabajo secundario, alquiler de habitación, etc.).

8.2 Categorías de gasto — diseño completo

Categoría	Emoji	Tipo	Subcategorías sugeridas	Integración FoodOS
Vivienda	■	Fija	Alquiler, hipoteca, comunidad, seguro hogar	Ninguna
Suministros	■	Variable	Electricidad, agua, gas, internet, móvil	Ninguna
Comida	■	Variable	Supermercado, frutería, carnicería, online	★ Calculada por FoodOS
Transporte	■	Variable	Bono bus, gasolina, parking, Uber, taxi	Ninguna
Suscripciones	■	Fija	Netflix, Spotify, gym, apps, software	Ninguna
Ocio	■	Variable	Restaurantes, cines, conciertos, viajes	Ninguna
Salud	■	Variable	Farmacia, médico, dentista, suplementos	Integrable con plan nutricional
Ropa	■	Variable	Ropa, zapatos, accesorios	Ninguna
Formación	■	Variable	Cursos, libros, certificaciones	Ninguna
Ahorro	■	Calculado	Colchón, inversión, objetivos específicos	★ Meta automática del plan
Otros	■	Variable	Cualquier gasto no categorizado	Ninguna

8.3 Conexión bancaria PSD2 — Nordigen (gratis)

La directiva PSD2 de la Unión Europea (en vigor desde 2018) obliga a los bancos a proporcionar APIs de acceso a datos de transacciones a terceros autorizados. Nordigen (ahora parte de GoCardless) es un proveedor certificado con acceso a más de 400 bancos europeos, incluyendo todos los principales bancos españoles.

Banco	Compatibilidad Nordigen
ING Direct España	✓ Completo (transacciones + saldos)
BBVA	✓ Completo
Santander	✓ Completo
CaixaBank	✓ Completo
Banco Sabadell	✓ Completo
Bankinter	✓ Completo
N26	✓ Completo
Revolut	✓ Completo (transacciones)

Banco	Compatibilidad Nordigen
Openbank	✓ Completo

Flujo técnico de autenticación PSD2:

#	Paso	Sistema responsable
1	Usuario pulsa 'Conectar banco' y selecciona su entidad	FoodOS Frontend
2	FoodOS llama a Nordigen API para crear una requisición (requisition_id)	FoodOS Backend → Nordigen
3	Nordigen devuelve una URL de autenticación del banco	Nordigen → FoodOS
4	FoodOS redirige al usuario a la URL del banco (popup o webview)	FoodOS Frontend
5	Usuario se autentica en su banco con sus credenciales	Usuario → Banco
6	El banco genera un token y lo envía a Nordigen	Banco → Nordigen
7	Nordigen notifica a FoodOS vía webhook con el account_id	Nordigen → FoodOS API
8	FoodOS guarda el account_id y begins syncing transacciones	FoodOS Backend
9	Reautorización obligatoria cada 90 días (norma PSD2)	FoodOS → Usuario

LÍMITES Y SEGURIDAD NORDIGEN

IMPORTANTE: Nordigen free plan permite hasta 50 conexiones bancarias activas simultáneas. Para un MVP y fase inicial de validación es más que suficiente. El plan de pago escala por número de usuarios y uso de la API. Los datos bancarios (tokens, account IDs) se almacenan en la tabla bank_connections de Supabase — NUNCA las credenciales del banco.

8.4 Importación alternativa CSV / PDF / Foto ticket

Para usuarios sin conexión bancaria, o como complemento, FoodOS ofrece tres métodos alternativos de importación de movimientos:

- **CSV del banco:** todos los bancos permiten exportar el extracto en .csv o .xlsx. FoodOS parsea el archivo, detecta columnas automáticamente y categoriza con IA.
- **PDF del extracto:** se sube el PDF y Gemini extrae el texto estructurado, identifica las transacciones y las importa.
- **Foto del ticket de compra:** Gemini Vision extrae los productos, cantidades y precios del ticket fotográfico. Los productos detectados se pueden añadir también al inventario.

8.6 Proyección de ahorro e interés compuesto

```
// utils/savings.ts – Proyecciones de ahorro

export interface SavingsProjection {

  months6: number;

  year1: number;

  years5_bank: number;    // cuenta corriente (0% interés)
```



```
years5_fund: number;    // fondo indexado (~7% anual estimado)

years10_fund: number;

emergencyFundMonths: number; // cuándo alcanza 3 meses de gastos
}

export function projectSavings(
  monthlyAmount: number,
  monthlyExpenses: number,
  annualRate = 0.07
): SavingsProjection {
  const fv = (m: number, n: number, r: number) =>
    r === 0 ? m * n : m * ((Math.pow(1 + r/12, n) - 1) / (r/12));

  return {
    months6:      Math.round(fv(monthlyAmount, 6, 0)),
    year1:        Math.round(fv(monthlyAmount, 12, 0)),
    years5_bank:  Math.round(fv(monthlyAmount, 60, 0)),
    years5_fund:  Math.round(fv(monthlyAmount, 60, annualRate)),
    years10_fund: Math.round(fv(monthlyAmount, 120, annualRate)),
    emergencyFundMonths: Math.ceil((monthlyExpenses * 3) / monthlyAmount),
  };
}

// Disclaimer: el 7% es una estimación histórica del S&P 500.
// FoodOS muestra siempre el disclaimer 'rentabilidad pasada no garantiza futura'
```

9

Módulo 6 — Nutrición y Objetivos Corporales

9.1 Perfil físico del usuario

Al configurar por primera vez el módulo de nutrición, FoodOS solicita los datos necesarios para calcular las necesidades calóricas del usuario. Todos los campos son editables en cualquier momento desde la configuración.

Campo	Tipo	Obligatorio	Para qué se usa
Edad	Integer (años)	Sí	Fórmula TMB
Sexo biológico	Enum (hombre/mujer)	Sí	Fórmula TMB (+5 hombre, -161 mujer)
Altura	Decimal (cm)	Sí	Fórmula TMB
Peso actual	Decimal (kg)	Sí	Fórmula TMB + cálculo proteína
% Grasa corporal	Decimal (%) nullable	No	Afinar proteína (usar masa magra)
Nivel de actividad	Enum (5 niveles)	Sí	Multiplicador TDEE
Objetivo corporal	Enum (4 opciones)	Sí	Ajuste calórico y macros
Días gym/semana	Integer[]	Sí	Ciclado calórico
Días concretos de gym	Array of weekdays	No	Saber hoy si es día gym o descanso
Alergias/intolerancias	Array de strings	No	Filtrar recetas incompatibles
Alimentos excluidos	Array de strings	No	No sugerir esos alimentos

9.2 Cálculo de TMB y TDEE — Fórmula Mifflin-St Jeor

```
// utils/nutrition-calc.ts

export function calcTMB(weight: number, height: number, age: number, sex: 'male'|'female'): number {

  // Fórmula Mifflin-St Jeor (la más precisa sin pruebas de laboratorio)

  const base = (10 * weight) + (6.25 * height) - (5 * age);

  return sex === 'male' ? base + 5 : base - 161;

}

export const ACTIVITY_FACTORS = {

  sedentary:      1.2,    // Sin ejercicio, trabajo de oficina

  light:          1.375, // Ejercicio 1-3 días/semana

  moderate:       1.55,   // Ejercicio 3-5 días/semana ← más común

  active:         1.725,  // Ejercicio intenso 6-7 días/semana

  very_active:    1.9,    // Trabajo físico + ejercicio diario

} as const;

export function calcTDEE(tmb: number, activityLevel: keyof typeof ACTIVITY_FACTORS): number {
```

```
    return Math.round(tmb * ACTIVITY_FACTORS[activityLevel]);
  }

  // Ejemplo – Unai: 82kg, 178cm, 24 años, hombre, moderado
  // TMB = (10×82) + (6.25×178) - (5×24) + 5 = 820 + 1112.5 - 120 + 5 = 1817.5 ≈ 1.818 kcal
  // TDEE = 1818 × 1.55 = 2.818 kcal/día
```

9.3 Modos de objetivo corporal — detalle completo

Modo	Ajuste TDEE	Proteína	Carbos	Grasas	Velocidad de cambio esper
Pérdida de grasa	−400 kcal	1.8g/kg	Moderados	25–30%	−0.5kg grasa/semana
Ganancia muscular	+250 kcal	2.0g/kg	Altos	25%	+0.3kg músculo/semana
Recomposición	±0 media +100 gym −200 rest	2.2g/kg	Ciclados	25%	Lenta: meses, sostenible
Mantenimiento	= TDEE	1.6g/kg	Ajuste	30%	Sin cambio esperado

9.4 Ciclado calórico para recomposición corporal

La recomposición corporal utiliza un sistema de ciclado calórico: los días de entrenamiento de fuerza se come en ligero superávit (+100 kcal sobre TDEE) para maximizar la síntesis de proteína muscular. Los días de descanso se come en ligero déficit (−200 kcal) para favorecer la oxidación de grasa. La media semanal resulta en un balance prácticamente neutro.

```
// utils/nutrition-calc.ts – Ciclado calórico recomposición

export function calcRecompTargets(
  tdee: number,
  weight: number,
  isGymDay: boolean
): DailyTargets {
  const protein_g = Math.round(weight * 2.2); // 2.2g por kg de peso
  const kcal = isGymDay ? tdee + 100 : tdee - 200;

  // Proteína fija siempre. Resto distribuido entre carbos y grasas.
  const protein_kcal = protein_g * 4;
  const remaining_kcal = kcal - protein_kcal;

  const fat_kcal = Math.round(kcal * 0.25); // 25% del total en grasas
  const carb_kcal = remaining_kcal - fat_kcal;

  return {
```

```
    kcal,

    protein_g,

    carbs_g: Math.round(carb_kcal / 4),

    fat_g:    Math.round(fat_kcal / 9),

    day_type: isGymDay ? 'gym' : 'rest'

  };

}
```

9.7 Ajuste automático de porciones en recetas

Cuando el usuario consulta recetas para una comida, FoodOS conoce exactamente qué macros le quedan por consumir ese día. El sistema calcula automáticamente la porción en gramos de cada ingrediente de la receta para que la comida complete (o se acerque lo máximo posible) a los macros pendientes, priorizando siempre la proteína.

9.8 Optimizador de fuentes proteicas por coste

Alimento (100g)	Proteína (g)	Precio típico	Coste por g proteína	Valoración
Pechuga de pollo	31g	€0,84/100g	€0,027/g prot	★ Óptimo
Atún en lata	28g	€0,78/100g	€0,028/g prot	★ Óptimo
Huevos (1 huevo=50g)	13g	€0,25/ud	€0,019/g prot	★★ Mejor
Pechuga de pavo	29g	€0,90/100g	€0,031/g prot	Bueno
Salmón	25g	€1,80/100g	€0,072/g prot	Caro
Tofu firme	17g	€0,60/100g	€0,035/g prot	Bueno (vegano)
Lentejas cocidas	9g	€0,15/100g	€0,017/g prot	★★ Mejor (fibra++)
Queso cottage	11g	€0,96/100g	€0,087/g prot	Muy caro
Solomillo de cerdo	22g	€1,45/100g	€0,066/g prot	Caro
Proteína whey (scoop)	24g	€0,90/scoop	€0,038/g prot	Bueno (suplemento)

10 Integraciones Cruzadas entre Módulos

La ventaja competitiva de FoodOS es la comunicación en tiempo real entre todos sus módulos. A continuación se detallan las 6 integraciones principales con su implementación técnica.

10.1 Inventario → Nutrición

Al añadir un alimento al inventario, sus macros se almacenan en la tabla `inventory_items`. Cuando el usuario registra una comida en el `food_log` (ej: 'He comido un filete de la nevera'), FoodOS busca automáticamente los macros del ítem en el inventario y los registra sin que el usuario tenga que introducirlos de nuevo. Cero doble entrada.

10.2 Carrito → Finanzas

Al completar la compra, se ejecuta una transacción atómica en Supabase que: (1) inserta una fila en la tabla `gastos` con `category='food'`, `source='foodos_cart'` y el importe total; (2) inserta todos los productos en `inventory_items`; (3) archiva el carrito. Si el banco está conectado y detecta un cargo del mismo supermercado ese día ($\pm 5\%$ importe), el sistema reconcilia automáticamente marcando la transacción bancaria como ya registrada.

10.3 Recetas → Presupuesto

Cada receta tiene un `coste_por_racion` calculado con los precios de Open Food Facts. El módulo financiero usa el historial de recetas cocinadas para calcular el presupuesto real de comida del mes. La IA compara este gasto real con el presupuesto establecido y puede sugerir recetas más económicas cuando el usuario está próximo al límite.

10.4 Banco → Inventario

Si las transacciones bancarias importadas incluyen un cargo de supermercado (Mercadona, Lidl, Carrefour, Dia, Alcampo...) y el usuario no ha actualizado el inventario ese día, FoodOS genera una notificación push: '¿Has hecho la compra hoy? Actualiza tu inventario.' Con un toque, puede subir foto del ticket para que la IA extraiga los productos automáticamente.

10.5 Nutrición → Carrito

Al analizar los macros de la semana, si el módulo de nutrición detecta que el usuario va a quedarse corto en proteína (por ejemplo, ha consumido menos del 80% del objetivo proteico los últimos 3 días), FoodOS genera una sugerencia en el carrito: las 3 fuentes proteicas más baratas por gramo de proteína disponibles en su supermercado más cercano.

10.6 Finanzas → Recetas

Cuando el usuario está a menos de €30 del límite mensual de presupuesto de comida (calculado en tiempo real), las recetas sugeridas por IA se filtran automáticamente para priorizar las que tienen el menor coste por ración, manteniendo siempre el cumplimiento del objetivo de macros del día.

11

Inteligencia Artificial — Implementación Completa

11.1 Opción A — Gemini 1.5 Flash (GRATIS)

Gemini 1.5 Flash es el modelo recomendado para FoodOS porque es el más generoso en cuota gratuita de todos los LLMs multimodales disponibles. Accesible a través de Google AI Studio sin tarjeta de crédito.

Característica	Gemini 1.5 Flash (gratis)
Requests por día	1.500 requests/día (gratuito)
Tokens por minuto	1.000.000 TPM
Tokens de contexto	1.000.000 tokens por request
Visión (imagen en el prompt)	✓ Incluido sin coste adicional
Multimodalidad	Texto + imagen en la misma llamada
Velocidad	Muy rápido (~1–3 segundos típico)
Precio al superar cuota	\$0.075 por millón tokens input / \$0.30 output
Necesita tarjeta	NO en Google AI Studio (desarrollo)
Para producción	Google Cloud Vertex AI (mismos precios, más control)

11.2 Opción B — OpenAI GPT-4o Vision (DE PAGO)

■■ IMPORTANTE — OPCIÓN DE PAGO

OpenAI NO tiene plan gratuito en producción. Requiere tarjeta de crédito para obtener API key. El plan de pago es por uso (pay-as-you-go). Se documenta aquí como alternativa premium para cuando FoodOS escale y necesite capacidades superiores.

Característica	GPT-4o (de pago)	GPT-4o-mini (más barato)
Precio input	\$5 / millón tokens	\$0.15 / millón tokens
Precio output	\$15 / millón tokens	\$0.60 / millón tokens
Visión	✓ (\$5/1M tokens imagen)	✓ (mismo precio)
Calidad detección food	Alta	Media-alta
Velocidad	Moderada	Rápida
Context window	128.000 tokens	128.000 tokens
Sin plan gratuito	Correcto, siempre de pago	Correcto
Estimación FoodOS	~\$15–50/mes a escala	~\$2–8/mes a escala

11.3 Comparativa y recomendación de estrategia

Criterio	Gemini 1.5 Flash	GPT-4o-mini	GPT-4o
Coste en desarrollo	€0	€0 (créditos iniciales)	€0 (créditos)
Coste en producción MVP	€0	~€2/mes	~€15/mes
Coste a escala (1k users)	~€5–15/mes	~€15–30/mes	~€80–150/mes
Detección de alimentos	Muy buena	Buena	Excelente
Recetas personalizadas	Excelente	Buena	Excelente
Categorización gastos	Excelente	Excelente	Excelente
Contexto largo (inventario completo)	✓ 1M tokens	✓ 128k tokens	✓ 128k tokens
Sin tarjeta de crédito	✓ (dev)	✗	✗
Recomendación	★ USAR ESTO	Alternativa económica	Solo si hay presupuesto

ESTRATEGIA DE IA RECOMENDADA

ESTRATEGIA RECOMENDADA: Usar Gemini 1.5 Flash en todas las fases hasta llegar a un presupuesto que justifique GPT-4o-mini. La diferencia de calidad en los casos de uso de FoodOS (detección de alimentos, recetas, categorización de gastos) es marginal, pero la diferencia de coste es enorme. Implementar ambos con una variable de entorno `AI_PROVIDER='gemini' | 'openai'` para poder cambiar sin tocar código.

11.4 Casos de uso de IA en FoodOS

Caso de uso	Input al modelo	Output esperado	Est. req/usuario/día	Coste (Gemini)
Detectar alimento por foto	Imagen JPG del producto	JSON: nombre, cantidad, macros, caducidad	1-5	€0
Recomendar recetas del día	Inventario completo + macros pendientes	Lista 3–5 recetas con porciones	1–3	€0
Categorizar transacción bancaria	Descripción del movimiento (texto)	Categoría + subcategoría	5–20	€0
Ajustar porciones receta	Macros pendientes + macros receta	Gramos por ingrediente	2–5	€0
Sugerir cena para cerrar macros	Macros consumidos + objetivo del día	Alimento concreto + gramos	1	€0
Parsear ticket de compra foto	Imagen del ticket	Lista productos + precios	0–2	€0
Estimar precio del carrito	Lista de productos + cantidades	Precio total estimado €	0–1	€0
Generar consejos de ahorro	Historial gastos + objetivos	3–5 consejos personalizados	0–1	€0
Detectar patrón de desperdicio	Historial caducidades + consumo	Insight + recomendación	0–1	€0

Total estimado por usuario activo: 12–38 requests/día. Con el free tier de 1.500 req/día, FoodOS puede soportar aproximadamente 40–125 usuarios activos diarios sin ningún coste.

11.5 Prompts detallados — Casos principales

Prompt 1 — Detección de alimento por foto:

```
const FOOD_DETECTION_PROMPT = `
```

Eres un experto en nutrición y ciencia de alimentos. Analiza la imagen y responde ÚNICAMENTE con un JSON válido, sin texto adicional ni markdown.

Schema requerido:

```
{
  "name": "nombre del alimento en español (ej: Pechuga de pollo)",
  "name_en": "nombre en inglés para búsqueda en APIs",
  "quantity_estimate": número (en la unidad indicada),
  "unit": "g | ml | ud | kg | L",
  "confidence": número entre 0 y 1,
  "macros_per_100g": {
    "kcal": número,
    "protein_g": número,
    "carbs_g": número,
    "fat_g": número,
    "fiber_g": número | null
  },
  "shelf_life_days": {
    "fridge": número,
    "freezer": número,
    "pantry": número | null
  },
  "allergens": ["gluten", "lactosa", ...] | [],
  "barcode_visible": boolean
}
```

Si no puedes identificar el alimento con confianza > 0.5, devuelve:

```
{ "name": null, "confidence": 0, "error": "No se puede identificar" }`;
```

Prompt 2 — Recomendación de recetas:

```
const buildRecipePrompt = (
  inventory: InventoryItem[],
  pendingMacros: DailyTargets,
  budget: number,
  preferences: UserPreferences
) => `
```

Eres el asistente nutricional de FoodOS. Recomienda recetas para la próxima comida.

INVENTARIO DISPONIBLE:

```
${inventory.map(i => `- ${i.name}: ${i.quantity}${i.unit} (caduca: ${i.expiryDate ?? 'no aplica'})`).join('\n')}
```

MACROS PENDIENTES HOY (\${preferences.dayType === 'gym' ? 'DÍA DE GYM' : 'DÍA DE DESCANSO'}):

- Calorías restantes: \${pendingMacros.kcal} kcal
- Proteína: \${pendingMacros.protein_g}g ← PRIORIDAD MÁXIMA
- Carbohidratos: \${pendingMacros.carbs_g}g
- Grasas: \${pendingMacros.fat_g}g

PRESUPUESTO COMIDA RESTANTE: €\${budget.toFixed(2)}

RESTRICCIONES:

- Alimentos excluidos: \${preferences.excluded.join(', ') || 'ninguno'}
- Alergias: \${preferences.allergies.join(', ') || 'ninguna'}

Responde con JSON array de 3 recetas:

```
[{
  "name": string,
  "ingredients_with_portions": [{ "name": string, "grams": number, "in_inventory": boolean }],
  "missing_ingredients": [{ "name": string, "quantity": string }],
  "macros_total": { "kcal": number, "protein_g": number, "carbs_g": number, "fat_g": number },
  "cost_estimate_eur": number,
  "prep_minutes": number,
  "uses_expiring": boolean,
  "why_recommended": string // 1 frase explicando la recomendación
}]`;
```

11.6 Gestión de límites y rate limiting

Con 1.500 requests/día en Gemini gratis, FoodOS necesita gestionar inteligentemente las llamadas para no agotar la cuota:

- **Caché de resultados:** las detecciones de productos por barcode se cachean en Supabase para no repetir llamadas al mismo EAN. La primera detección es gratuita, las siguientes usan la caché.
- **Rate limiting por usuario:** cada usuario tiene un máximo de 50 llamadas IA/día. En Supabase se lleva un contador diario por user_id.
- **Fallback automático:** si Gemini devuelve error 429 (quota exceeded), la app muestra un formulario manual con autocompletado de Open Food Facts (sin IA).
- **Batching:** la categorización de transacciones bancarias se hace en batch (todas las del día en una sola llamada) en vez de una por transacción.

- **Queue de baja prioridad:** consejos de ahorro y análisis de desperdicio se generan en segundo plano, no en tiempo real.

12 Modelo de Datos Completo

12.1 Esquema SQL de todas las tablas principales

users — Gestionado por Supabase Auth

```
-- Supabase Auth genera esta tabla automáticamente

-- Solo se muestra para referencia

CREATE TABLE auth.users (

  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),

  email       TEXT UNIQUE NOT NULL,

  created_at  TIMESTAMPTZ DEFAULT now()

);
```

user_profiles

```
CREATE TABLE public.user_profiles (

  user_id      UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,

  full_name    TEXT,

  avatar_url   TEXT,

  age          INTEGER CHECK (age > 0 AND age < 120),

  sex          TEXT CHECK (sex IN ('male', 'female')),

  height_cm    DECIMAL(5,1),

  weight_kg    DECIMAL(5,1),

  body_fat_pct DECIMAL(4,1),          -- nullable, opcional

  activity_level TEXT CHECK (activity_level IN

    ('sedentary', 'light', 'moderate', 'active', 'very_active')),

  goal         TEXT CHECK (goal IN

    ('fat_loss', 'muscle_gain', 'recomp', 'maintain')),

  gym_days     INTEGER[] DEFAULT '{}', -- 0=Dom, 1=Lun ... 6=Sáb

  excluded_foods TEXT[] DEFAULT '{}',

  allergies    TEXT[] DEFAULT '{}',

  push_token   TEXT,                  -- Expo push token

  bank_sync_enabled BOOLEAN DEFAULT false,

  updated_at   TIMESTAMPTZ DEFAULT now()

);
```

almacenes

```
CREATE TABLE public.almacenes (  
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    owner_id    UUID NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,  
    name        TEXT NOT NULL,  
    type        TEXT CHECK (type IN ('fridge', 'freezer', 'pantry', 'custom'))  
                DEFAULT 'custom',  
    emoji       TEXT DEFAULT '■■■',  
    is_shared    BOOLEAN DEFAULT false,  
    created_at  TIMESTAMPTZ DEFAULT now()  
);  
  
CREATE TABLE public.almacen_members (  
    almacen_id  UUID REFERENCES public.almacenes(id) ON DELETE CASCADE,  
    user_id     UUID REFERENCES auth.users(id) ON DELETE CASCADE,  
    role        TEXT CHECK (role IN ('owner', 'member')) DEFAULT 'member',  
    joined_at   TIMESTAMPTZ DEFAULT now(),  
    PRIMARY KEY (almacen_id, user_id)  
);
```

inventory_items

```
CREATE TABLE public.inventory_items (  
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    almacen_id  UUID NOT NULL REFERENCES public.almacenes(id) ON DELETE CASCADE,  
    owner_id    UUID NOT NULL REFERENCES auth.users(id),  
    name        TEXT NOT NULL,  
    barcode     TEXT,  
    quantity    DECIMAL(10,2) NOT NULL DEFAULT 1,  
    unit        TEXT DEFAULT 'ud',          -- g, ml, ud, kg, L  
    expiry_date DATE,  
    image_url   TEXT,  
    kcal_per_100 DECIMAL(7,2),  
    protein_per_100 DECIMAL(6,2),  
    carbs_per_100 DECIMAL(6,2),  
    fat_per_100  DECIMAL(6,2),  
    fiber_per_100 DECIMAL(6,2),  
    price_estimate DECIMAL(8,2),
```

```
is_cooked      BOOLEAN DEFAULT false,    -- tupper/preparado
open_food_id   TEXT,                      -- ID en Open Food Facts
added_at       TIMESTAMPTZ DEFAULT now(),
updated_at     TIMESTAMPTZ DEFAULT now()
);
```

recipes

```
CREATE TABLE public.recipes (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  author_id       UUID NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,
  title           TEXT NOT NULL,
  description      TEXT,
  image_url       TEXT,
  video_url       TEXT,                  -- Cloudinary URL
  video_thumbnail TEXT,                  -- Cloudinary thumbnail
  prep_minutes    INTEGER DEFAULT 0,
  servings        INTEGER DEFAULT 1,
  difficulty       TEXT CHECK (difficulty IN ('easy','medium','hard')),
  is_public       BOOLEAN DEFAULT false,
  tags            TEXT[] DEFAULT '{}',
  kcal_per_serving DECIMAL(7,2),
  protein_per_serving DECIMAL(6,2),
  carbs_per_serving DECIMAL(6,2),
  fat_per_serving  DECIMAL(6,2),
  cost_per_serving DECIMAL(8,2),
  likes_count     INTEGER DEFAULT 0,
  saves_count     INTEGER DEFAULT 0,
  created_at      TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE public.recipe_ingredients (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  recipe_id       UUID NOT NULL REFERENCES public.recipes(id) ON DELETE CASCADE,
  name            TEXT NOT NULL,
  quantity        DECIMAL(10,2),
  unit            TEXT,
```

```
kcal_per_100    DECIMAL(7,2),  
protein_per_100 DECIMAL(6,2),  
carbs_per_100   DECIMAL(6,2),  
fat_per_100     DECIMAL(6,2),  
sort_order     INTEGER DEFAULT 0  
);
```

food_log

```
CREATE TABLE public.food_log (  
    id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    user_id       UUID NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,  
    log_date      DATE NOT NULL DEFAULT CURRENT_DATE,  
    meal_type     TEXT CHECK (meal_type IN ('breakfast','lunch','dinner','snack')),  
    item_name     TEXT NOT NULL,  
    quantity_g    DECIMAL(8,2),  
    kcal          DECIMAL(7,2),  
    protein_g     DECIMAL(6,2),  
    carbs_g       DECIMAL(6,2),  
    fat_g         DECIMAL(6,2),  
    source        TEXT CHECK (source IN ('manual','recipe','inventory','barcode')),  
    source_id     UUID,                -- recipe_id o inventory_item_id  
    created_at    TIMESTAMPTZ DEFAULT now()  
);  
  
-- Índice para consultas rápidas del día actual  
  
CREATE INDEX idx_food_log_user_date ON public.food_log(user_id, log_date);
```

gastos

```
CREATE TABLE public.gastos (  
    id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    user_id       UUID NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,  
    amount        DECIMAL(10,2) NOT NULL,  
    description   TEXT,  
    category      TEXT NOT NULL,  
    subcategory   TEXT,  
    frequency     TEXT CHECK (frequency IN ('once','weekly','monthly','yearly'))
```

```

        DEFAULT 'once',

    txn_date        DATE NOT NULL DEFAULT CURRENT_DATE,

    source          TEXT CHECK (source IN ('manual','bank','foodos_cart','csv','pdf','photo'))

        DEFAULT 'manual',

    bank_txn_id     TEXT UNIQUE,          -- para deduplicación con banco

    bank_account_id TEXT,

    notes          TEXT,

    created_at      TIMESTAMPTZ DEFAULT now()

);

CREATE INDEX idx_gastos_user_date ON public.gastos(user_id, txn_date);

CREATE INDEX idx_gastos_category ON public.gastos(user_id, category);

```

ingresos_fuentes + objetivos_ahorro

```

CREATE TABLE public.ingresos_fuentes (

    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),

    user_id         UUID NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,

    name           TEXT NOT NULL,

    amount         DECIMAL(10,2) NOT NULL,

    frequency       TEXT CHECK (frequency IN ('weekly','biweekly','monthly','yearly')),

    day_of_month    INTEGER CHECK (day_of_month BETWEEN 1 AND 31),

    active          BOOLEAN DEFAULT true,

    created_at      TIMESTAMPTZ DEFAULT now()

);

CREATE TABLE public.objetivos_ahorro (

    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),

    user_id         UUID NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,

    name           TEXT NOT NULL,

    emoji          TEXT DEFAULT '■',

    target_amount   DECIMAL(10,2) NOT NULL,

    current_amount  DECIMAL(10,2) DEFAULT 0,

    target_date     DATE,

    is_completed    BOOLEAN DEFAULT false,

    created_at      TIMESTAMPTZ DEFAULT now()

);

```

bank_connections

```
CREATE TABLE public.bank_connections (  
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id           UUID NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,  
  provider          TEXT DEFAULT 'nordigen',  
  institution_id    TEXT NOT NULL,      -- ID del banco en Nordigen  
  institution_name  TEXT,  
  requisition_id    TEXT NOT NULL,      -- ID de la requisición PSD2  
  account_ids       TEXT[] DEFAULT '{}',  
  status            TEXT CHECK (status IN ('active','expired','pending','revoked'))  
                    DEFAULT 'pending',  
  last_sync         TIMESTAMPTZ,  
  expires_at        TIMESTAMPTZ,       -- máximo 90 días por PSD2  
  created_at        TIMESTAMPTZ DEFAULT now()  
);  
  
-- Nunca se almacenan credenciales bancarias ni tokens de acceso del banco.  
-- Solo los IDs de Nordigen para hacer consultas de lectura.  
COMMENT ON TABLE public.bank_connections IS  
  'Solo IDs de Nordigen. Credenciales bancarias NUNCA se almacenan aquí.';
```


13 APIs Externas — Documentación de Uso

13.1 Open Food Facts API

Open Food Facts es una base de datos colaborativa y open source con más de 3 millones de productos alimenticios. Su API es completamente gratuita y sin límite de uso (se pide usar un User-Agent identificativo por buenas prácticas).

```
// lib/open-food-facts.ts

const BASE_URL = 'https://world.openfoodfacts.org/api/v2';

const USER_AGENT = 'FoodOS/1.0 (foodos.app; contacto@foodos.app)';

// Buscar por código de barras

export async function getProductByBarcode(barcode: string): Promise<OFFProduct | null> {

  const res = await fetch(`${BASE_URL}/product/${barcode}.json`, {

    headers: { 'User-Agent': USER_AGENT }

  });

  const data = await res.json();

  if (data.status !== 1) return null;

  return {

    name:      data.product.product_name_es || data.product.product_name,

    brand:     data.product.brands,

    imageUrl:  data.product.image_front_url,

    kcal:      data.product.nutriments?.['energy-kcal_100g'],

    protein:   data.product.nutriments?.proteins_100g,

    carbs:    data.product.nutriments?.carbohydrates_100g,

    fat:       data.product.nutriments?.fat_100g,

    fiber:     data.product.nutriments?.fiber_100g,

    allergens: data.product.allergens_tags ?? [],

  };

}

// Búsqueda por texto (autocompletado)

export async function searchProducts(query: string, limit = 10): Promise<OFFProduct[]> {

  const params = new URLSearchParams({

    search_terms: query,

    search_simple: '1',

    action: 'process',

  });
```

```
    json: '1',

    page_size: String(limit),

    fields: 'product_name_es,product_name,brands,nutriments,image_front_url,code'
  });

const res = await fetch(`https://world.openfoodfacts.org/cgi/search.pl?${params}`,
  { headers: { 'User-Agent': USER_AGENT } });

const data = await res.json();

return data.products?.map(mapOFFProduct) ?? [];
}
```

13.2 Nordigen / GoCardless — PSD2

```
// lib/nordigen.ts – Cliente para la API bancaria PSD2

const NORDIGEN_BASE = 'https://ob.nordigen.com/api/v2';

// 1. Obtener token de acceso (válido 24h)

export async function getNordigenToken(): Promise<string> {

  const res = await fetch(`${NORDIGEN_BASE}/token/new/`, {

    method: 'POST',

    headers: { 'Content-Type': 'application/json' },

    body: JSON.stringify({

      secret_id: process.env.NORDIGEN_SECRET_ID,

      secret_key: process.env.NORDIGEN_SECRET_KEY

    })

  });

  const data = await res.json();

  return data.access; // JWT de 24h
}

// 2. Crear requisición (autorización del usuario con su banco)

export async function createRequisition(

  institutionId: string,

  redirectUrl: string,

  token: string

): Promise<{ id: string; link: string }> {

  const res = await fetch(`${NORDIGEN_BASE}/requisitions/`, {

    method: 'POST',
```

```
headers: { 'Authorization': `Bearer ${token}`, 'Content-Type': 'application/json' },
body: JSON.stringify({
  redirect: redirectUrl,
  institution_id: institutionId,
  reference: crypto.randomUUID(),
  agreement: null,
  user_language: 'ES'
})
});

return res.json(); // { id, link } – link es la URL a la que redirigir al usuario
}

// 3. Obtener transacciones (una vez autorizado)
export async function getTransactions(
  accountId: string,
  token: string,
  dateFrom?: string // YYYY-MM-DD
): Promise<NordigenTransaction[]> {
  const params = dateFrom ? `?date_from=${dateFrom}` : '';
  const res = await fetch(`${NORDIGEN_BASE}/accounts/${accountId}/transactions/${params}`, {
    headers: { 'Authorization': `Bearer ${token}` }
  });
  const data = await res.json();
  return data.transactions?.booked ?? [];
}
```

14

Roadmap y Fases de Desarrollo

14.1 Fase 0 — Setup del proyecto (Semana 1)

OBJETIVO DE LA FASE

Objetivo: tener el proyecto configurado, corriendo en local y desplegado en Vercel con Supabase conectado. Al final de esta fase, el equipo (o el desarrollador solo) puede hacer un login básico y ver una pantalla vacía funcional.

- Crear repositorio en GitHub con estructura de monorepo (Turborepo)
- Inicializar apps/web con Next.js 14 y apps/mobile con Expo SDK 51
- Configurar TypeScript strict en todo el monorepo
- Crear proyecto en Supabase: habilitar Auth, crear tablas base con RLS
- Configurar Vercel: importar repo, variables de entorno
- Obtener API keys: Google AI Studio (Gemini), Nordigen sandbox, Cloudinary
- Configurar Firebase project para FCM (notificaciones push)
- Setup packages/types con las interfaces TypeScript principales
- Setup packages/api-client con el cliente de Supabase configurado
- Primer deploy funcional en Vercel con login básico

14.2 Fase 1 — MVP Core (Semanas 2–7)

Semana 2 — Inventario básico

- CRUD completo de almacenes y productos
- Integración Open Food Facts (búsqueda + barcode)
- Pantalla de inventario con filtros y búsqueda
- Caducidades automáticas por tipo de almacén

Semana 3 — Recetas

- CRUD de recetas con ingredientes
- Cálculo automático de macros de receta
- Semáforo de disponibilidad con inventario
- Botón 'Añadir faltantes al carrito'

Semana 4 — Carrito de compra

- Listas múltiples por tienda
- Merge automático de ingredientes duplicados
- Check de items con sincronización cruzada entre recetas
- Primer borrador de integración carrito → finanzas

Semana 5 — Pantalla de inicio y alertas

- Home con alertas de caducidad y stock bajo
- Mini-resumen financiero en home
- Notificaciones push básicas (caducidad)

- Navegación completa entre módulos

Semana 6 — Auth y almacén compartido

- Login/registro con Supabase Auth (email + Google)
- Invitar usuarios al almacén (por email o enlace)
- Notificación cuando alguien consume tu alimento
- Propietario por alimento en almacén compartido

Semana 7 — QA y pulido MVP

- Pruebas en dispositivos iOS y Android reales
- Corrección de bugs y ajustes de UX
- Optimización de queries (índices Supabase)
- Preparación para beta testers

14.3 Fase 2 — IA, Nutrición y Finanzas (Semanas 8–13)

Semana 8 — IA básica

- Detección de alimentos por foto (Gemini Vision)
- Recomendación de recetas con IA
- Categorización automática de gastos
- Rate limiting y caché de resultados IA

Semana 9 — Módulo nutrición

- Onboarding de perfil físico (4 pasos)
- Cálculo TMB/TDEE
- Configuración de objetivo corporal
- Distribución de macros por modo y por día

Semana 10 — Seguimiento diario

- Food log con registro por franja horaria
- Anillo de progreso de calorías
- Ajuste automático de porciones en recetas
- Sugerencia de cena para cerrar macros

Semana 11 — Finanzas básicas

- CRUD de gastos con categorías
- CRUD de fuentes de ingreso
- Presupuesto mensual por categorías
- Alerta de desviación de presupuesto

Semana 12 — Conexión bancaria

- Integración Nordigen PSD2
- Importación de transacciones automática
- Importación manual CSV/PDF
- Parseo de ticket de compra con foto

Semana 13 — Integraciones

- Carrito → finanzas (registro automático)
- Banco → inventario (sugerencia)
- Nutrición → carrito (compra por macros)

■ Proyección de ahorro con interés compuesto

14.4 Fase 3 — Social y Funcionalidades Avanzadas

- Feed social con vídeos (Cloudinary): publicar, descubrir, guardar recetas
- Optimizador de fuentes proteicas por coste con ranking actualizado
- Estadísticas unificadas: alimentación + finanzas en una sola pantalla con toggle
- Modo ahorro máximo: replanificación de compra con macros mantenidos
- Gamificación: rachas de días sin desperdiciar, logros, retos semanales
- Escaneo de ticket de compra para actualizar inventario masivamente
- Integración con Apple Health / Google Fit para importar actividad física
- Soporte para múltiples perfiles (ej: familia con diferentes objetivos)
- Hardware fase futura: integración con Raspberry Pi + lector barcode en nevera

14.5 Estimación de costes por escala de usuarios

Fase / Usuarios	Supabase	Vercel	Gemini IA	Nordigen	Cloudinary	Total/mes
Desarrollo (0 users)	€0	€0	€0	€0	€0	€0
MVP (<100 users)	€0	€0	€0	€0	€0	€0
Early (<500 users)	€0	€0	~€5	€0	€0	~€5
Growth (1.000 users)	€25	€0–20	~€30	~€30	€0	~€85
Scale (5.000 users)	€25+	€20	~€150	~€150	€0–89	~€400
Large (20.000 users)	€599	€20+	~€600	~€600	€89	~€1.900

14.6 Modelo de negocio y monetización

FoodOS puede operar como freemium. Las funcionalidades básicas son y serán siempre gratuitas. El plan premium desbloquea funciones avanzadas que tienen un coste real de infraestructura:

Funcionalidad	Plan gratuito	Plan premium (~2,99€/mes)
Inventario básico (1 almacén)	✓	✓
Recetas ilimitadas	✓	✓
Carrito de compra	✓	✓
Seguimiento nutricional básico	✓	✓
Gastos manuales	✓	✓
Almacenes ilimitados	3 máximo	✓ Ilimitados
Conexión bancaria PSD2	✗	✓ (cuesta €/usuario)
IA sin límite	50 req/día	✓ Ilimitado
Feed social y vídeos	Solo lectura	✓ Publicar

Funcionalidad	Plan gratuito	Plan premium (~2,99€/mes)
Estadísticas historial completo	30 días	✓ Sin límite
Almacén compartido	2 usuarios	✓ Ilimitados
Exportar datos (CSV/PDF)	✗	✓

Resumen Ejecutivo Final

ASPECTO	DETALLE COMPLETO
Plataformas	iOS · Android · Web (SSR + PWA) — mismo código base
Módulos	Inventario · Recetas · Carrito · Feed Social · Finanzas · Nutrición
Stack web	Next.js 14 (App Router) + TypeScript + TailwindCSS
Stack móvil	Expo SDK 51 + React Native + Expo Router
Base de datos	Supabase PostgreSQL con RLS + Realtime WebSockets
Autenticación	Supabase Auth (principal) + Firebase Auth (alternativa SMS)
Hosting web	Vercel Hobby (gratis: 100GB BW, serverless functions)
Storage	Supabase Storage (fotos) + Cloudinary (vídeos)
Notificaciones	Expo + Firebase FCM — completamente gratis
IA principal (GRATIS)	Gemini 1.5 Flash — 1.500 req/día, visión incluida, 1M ctx
IA alternativa (PAGO)	OpenAI GPT-4o-mini — \$0.15/1M tokens, requiere tarjeta
API alimentos	Open Food Facts — open source, sin límite, sin auth
API bancaria	Nordigen/GoCardless PSD2 — gratis hasta 50 usuarios banco
Monorepo	Turborepo — código compartido entre web y móvil
Coste MVP completo	€0/mes hasta aproximadamente 500 usuarios activos
Coste a 1.000 usuarios	~€85/mes (Supabase Pro + Nordigen + Gemini uso)
Modelo negocio	Freemium — plan premium ~€2,99/mes para funciones avanzadas
Tablas en DB	12 tablas principales con RLS, índices y relaciones completas
Fases desarrollo	Fase 0 (1 sem) + Fase 1 (6 sem) + Fase 2 (6 sem) + Fase 3
Tiempo MVP funcional	~7 semanas trabajando solo a ritmo sostenible

Módulo 7 — Búsqueda y Generación de Recetas por Ingrediente

Este módulo permite al usuario encontrar o generar recetas a partir de uno o más ingredientes que él mismo elige, con distintos niveles de restricción. El caso de uso central es: 'Tengo zanahoria y quiero recetas que la usen sí o sí', pero el sistema permite mucho más matiz que eso.

DIFERENCIADOR DEL MÓDULO

Diferencia clave con la búsqueda normal de recetas: en la búsqueda normal el usuario busca por nombre de receta o se le recomiendan en base a todo su inventario. En este módulo, el usuario elige explícitamente qué ingrediente protagoniza, y puede marcar otros como opcionales o excluidos.

15.1 Los cuatro modos de búsqueda

El módulo opera con cuatro modos seleccionables mediante pills en la parte superior de la pantalla. Cada modo aplica una lógica de filtrado diferente.

Modo	Lógica de filtrado	Cuándo usarlo	Resultado típico
Sí o sí lo lleva	El ingrediente marcado como obligatorio debe aparecer en la lista de ingredientes	Se debe aparecer en la lista de ingredientes	Recetas que incluyen el ingrediente obligatorio
Protagonista	El ingrediente obligatorio debe ser el primero en la lista de ingredientes	Quiero que el ingrediente sea el protagonista	Recetas donde el ingrediente es el protagonista
Solo con lo que tengo	El ingrediente es obligatorio Y el 100% de los demás ingredientes están en el inventario	Quiero recetas con ingredientes que yo tengo	Solo recetas ejecutables con los ingredientes disponibles
Generar con IA	Gemini recibe el ingrediente obligatorio y los opcionales/excluidos	Quiero que la IA genere recetas personalizadas	Recetas generadas por IA basadas en los ingredientes

15.2 Sistema de chips — estados de los ingredientes

El usuario puede añadir múltiples ingredientes al filtro y asignar a cada uno un estado. Pulsar sobre un chip cicla su estado en el orden: opcional → obligatorio → excluido → opcional.

Estado	Visual	Significado	Impacto en el filtrado
Obligatorio	Verde + candado	La receta sí o sí debe contener este ingrediente	WHERE ingredientes INCLUDES este item (obligatorio)
Opcional	Gris neutro	La receta puede o no contener el ingrediente	SIN filtro en el query (informativo)
Excluido	Rojo tachado	Ninguna receta puede contener este ingrediente	WHERE ingredientes NOT INCLUDES este item

Los chips pueden añadirse de dos formas: escribiendo en el buscador con autocompletado desde Open Food Facts, o pulsando 'Añadir desde mi nevera' para seleccionar directamente de los ingredientes disponibles en el inventario actual sin escribir nada.

15.3 Semáforo de disponibilidad en los resultados

Cada receta en los resultados muestra un desglose visual de sus ingredientes indicando cuáles tiene el usuario en su inventario y cuáles no.

Color del chip	Significado	Condición técnica
Verde + candado	Ingrediente obligatorio — siempre presente	Marcado como required por el usuario
Verde normal	Tienes suficiente en inventario	inventory_items WHERE name MATCH AND quantity >= requerida
Amarillo	Tienes pero puede que no sea suficiente	inventory_items WHERE name MATCH AND 0 < quantity < requerida
Rojo	No tienes este ingrediente	inventory_items WHERE name NOT FOUND OR quantity = 0

UX: INGREDIENTES FALTANTES

Al pulsar sobre cualquier ingrediente en rojo, se abre un modal con dos opciones: 'Añadir al carrito' (lo agrega al carrito de compra activo) o 'Ignorar' (marca ese ingrediente como prescindible para esta receta). Esto permite al usuario decidir si quiere comprar el ingrediente o sustituirlo.

15.4 Ordenación de resultados

Las recetas en los resultados se ordenan por relevancia combinada, no simplemente por presencia del ingrediente obligatorio. El algoritmo de puntuación es el siguiente:

```
// Algoritmo de puntuación y ordenación de recetas

// utils/recipe-relevance-score.ts

interface RecipeScore {
  recipe: Recipe;
  score: number;
  breakdown: {
    requiredPresent: number; // 0 o 100 (filtro duro, no llega aquí si es 0)
    ingredientRatio: number; // % de ingredientes disponibles en inventario (0-40 pts)
    protagonistBonus: number; // +20 si el ingrediente requerido es el primero o >30% peso
    expiryBonus: number; // +15 si usa ingredientes que caducan en <3 días
    macroFitBonus: number; // +25 si encaja en macros pendientes del día (±15%)
    costBonus: number; // +10 si coste por ración < media del usuario
  }
}

export function scoreRecipe(
  recipe: Recipe,
  requiredIngredients: string[],
  inventory: InventoryItem[],
  pendingMacros: DailyTargets | null,
  expiringItems: string[]
```

```
) : RecipeScore {  
  
  // 1. Filtro duro: ingredientes obligatorios (ya filtrado antes de llegar aquí)  
  
  const requiredPresent = 100;  
  
  // 2. Ratio de ingredientes disponibles (0-40 puntos)  
  
  const available = recipe.ingredients.filter(ing =>  
    inventory.some(item => item.name.toLowerCase().includes(ing.name.toLowerCase()) && item.quantity > 0  
  )  
  ).length;  
  
  const ingredientRatio = Math.round((available / recipe.ingredients.length) * 40);  
  
  // 3. Bonus protagonista (0 o +20)  
  
  const firstIng = recipe.ingredients[0]?.name.toLowerCase();  
  const reqLower = requiredIngredients[0]?.toLowerCase();  
  const isProtagonist = firstIng?.includes(reqLower) ||  
    (recipe.ingredients.find(i => i.name.toLowerCase().includes(reqLower))?.weightPercent ?? 0) > 30;  
  const protagonistBonus = isProtagonist ? 20 : 0;  
  
  // 4. Bonus caducidad (0 o +15)  
  
  const usesExpiring = recipe.ingredients.some(ing =>  
    expiringItems.some(exp => exp.toLowerCase().includes(ing.name.toLowerCase()))  
  );  
  const expiryBonus = usesExpiring ? 15 : 0;  
  
  // 5. Bonus macros (0 o +25)  
  
  let macroFitBonus = 0;  
  
  if (pendingMacros) {  
    const kcalFit = Math.abs(recipe.kcalPerServing - pendingMacros.kcal) / pendingMacros.kcal;  
    const protFit = Math.abs(recipe.proteinPerServing - pendingMacros.protein_g) / pendingMacros.protein_g;  
    if (kcalFit < 0.15 && protFit < 0.15) macroFitBonus = 25;  
    else if (kcalFit < 0.25) macroFitBonus = 10;  
  }  
  
  // 6. Bonus precio (0 o +10)  
  
  const costBonus = (recipe.costPerServing ?? 999) < 2.5 ? 10 : 0;  
  
  const score = ingredientRatio + protagonistBonus + expiryBonus + macroFitBonus + costBonus;
```

```
    return { recipe, score, breakdown: { requiredPresent, ingredientRatio, protagonistBonus, expiryBonus,
macroFitBonus, costBonus } };

}
```

15.5 Modo 4 — Generación de recetas con IA

El modo de generación con IA es el más potente. En vez de buscar en la base de datos, construye un prompt detallado para Gemini 1.5 Flash que incluye el ingrediente obligatorio, el inventario completo, los macros pendientes del día y las restricciones del usuario.

Flujo completo de generación:

#	Paso	Sistema	Datos involucrados
1	El usuario escribe 'zanahoria' y activa Frontend	Frontend	Ingrediente obligatorio + chips adicionales
2	Frontend llama a /api/ai/generate-recipe/route.ts	API Route	Ingrediente + inventario + macros pendientes + perfil
3	API construye el prompt y llama a Gemini 1.5 Flash	Backend	Prompt completo (ver §15.6)
4	Gemini devuelve 2–3 recetas en JSON	Gestor de Backend	Nombre, ingredientes con gramos, macros, pasos
5	Backend cruza ingredientes generados con inventario	Backend + Supabase	Marca has/missing por ingrediente
6	Respuesta llega al frontend con el schema de Frontend	Frontend	Recetas con disponibilidad y coste estimado
7	Usuario puede guardar la receta o añadirla al carrito	Frontend	INSERT en recipes + cart_items

15.6 Prompt completo — Generación de receta por ingrediente

```
// Endpoint completo de generación IA con contexto nutricional

// app/api/ai/generate-recipe/route.ts

import { NextRequest } from 'next/server';

import { createClient } from '@lib/supabase/server';

export async function POST(req: NextRequest) {

  const {

    requiredIngredients,    // string[] – ingredientes obligatorios (chips verdes)

    excludedIngredients,    // string[] – ingredientes excluidos (chips rojos)

    optionalIngredients,    // string[] – ingredientes opcionales (chips grises)

    mode,                   // 'mandatory' | 'protagonist' | 'inventory_only' | 'ai_generate'

    pendingMacros,          // { kcal, protein_g, carbs_g, fat_g } | null

    userId,

  } = await req.json();

  const supabase = createClient();
```

```
// Obtener inventario completo del usuario

const { data: inventory } = await supabase

  .from('inventory_items')

  .select('name, quantity, unit, expiry_date')

  .eq('user_id', userId)

  .gt('quantity', 0);

// Obtener perfil nutricional

const { data: profile } = await supabase

  .from('user_profiles')

  .select('goal, allergies, excluded_foods, weight_kg')

  .eq('user_id', userId)

  .single();

const expiringItems = inventory

  ?.filter(i => {

    if (!i.expiry_date) return false;

    const daysLeft = Math.ceil((new Date(i.expiry_date).getTime() - Date.now()) / 86400000);

    return daysLeft <= 3;

  })

  .map(i => i.name) ?? [];

const prompt = buildPrompt({

  requiredIngredients,

  excludedIngredients,

  optionalIngredients,

  inventory: inventory ?? [],

  expiringItems,

  pendingMacros,

  profile,

  mode

});

const geminiRes = await fetch(

  `https://generativelanguage.googleapis.com/v1beta/models/gemini-1.5-flash:generateContent?key=\${process.env.GEMINI_API_KEY}`,

  {
```

```
method: 'POST',

headers: { 'Content-Type': 'application/json' },

body: JSON.stringify({

  contents: [{ parts: [{ text: prompt }] }],

  generationConfig: { temperature: 0.4, maxOutputTokens: 2048 }

})

}

);

const geminiData = await geminiRes.json();

const raw = geminiData.candidates[0].content.parts[0].text;

const clean = raw.replace(/\\`\\`\\`json|\\`\\`\\`/g, '').trim();

const recipes = JSON.parse(clean);

// Cruzar con inventario para marcar disponibilidad

const enriched = recipes.map((r: any) => ({

  ...r,

  ingredients: r.ingredients.map((ing: any) => ({

    ...ing,

    inInventory: inventory?.some(item =>

      item.name.toLowerCase().includes(ing.name.toLowerCase()) && item.quantity > 0

    ) ?? false,

    isRequired: requiredIngredients.some(req =>

      ing.name.toLowerCase().includes(req.toLowerCase())

    ),

  })),

})));

return Response.json({ recipes: enriched });

}

function buildPrompt(ctx: PromptContext): string {

  const macroStr = ctx.pendingMacros

  ? `\\`MACROS PENDIENTES HOY (PRIORIDAD ALTA):

  - Calorías restantes: \\${ctx.pendingMacros.kcal} kcal

  - Proteína restante: \\${ctx.pendingMacros.protein_g}g (PRIORIDAD MÁXIMA)

  - Carbohidratos: \\${ctx.pendingMacros.carbs_g}g`
```

```

- Grasas: \${ctx.pendingMacros.fat_g}g\`

: 'Sin restricción calórica específica para hoy.';

const modeInstruction = ctx.mode === 'protagonist'

? \`El ingrediente "\${ctx.requiredIngredients[0]}" DEBE ser el primero en la lista y suponer más de
l 30% del peso total de la receta.\`

: \`La receta DEBE contener: \${ctx.requiredIngredients.join(', ')}.\`;

return \`Eres el chef IA de Food0S. Genera exactamente 2 recetas de cocina casera en español.

REGLAS ESTRUCTURADAS:

1. \${modeInstruction}

2. NUNCA usar estos ingredientes: \${[...ctx.excludedIngredients, ...(ctx.profile?.excluded_foods ?? []),
, ...(ctx.profile?.allergies ?? [])].join(', ') || 'ninguno'}

3. Prioriza ingredientes que caducan pronto: \${ctx.expiringItems.join(', ') || 'ninguno en riesgo'}

4. Usa preferentemente ingredientes del inventario disponible.

\${macroStr}

INVENTARIO DISPONIBLE:

\${ctx.inventory.slice(0, 40).map(i => \`- \${i.name}: \${i.quantity}\${i.unit}\`).join('\n')}`

Responde ÚNICAMENTE con este JSON (sin texto adicional ni markdown):

[
  {
    "name": "Nombre de la receta en español",
    "description": "Descripción apetitosa en 1 frase",
    "prep_minutes": número,
    "servings": número,
    "ingredients": [
      { "name": "nombre", "quantity": número, "unit": "g|ml|ud", "weight_percent": número }
    ],
    "steps": ["Paso 1...", "Paso 2...", "Paso 3..."],
    "macros_per_serving": { "kcal": número, "protein_g": número, "carbs_g": número, "fat_g": número },
    "cost_estimate_eur": número,
    "why_this_recipe": "Por qué esta receta encaja con el contexto del usuario"
  }
]`;
}

```

15.7 Persistencia de recetas generadas por IA

Las recetas generadas por IA son temporales hasta que el usuario decida guardarlas. Al pulsar 'Guardar receta', la receta se inserta en la tabla recipes con author_id del usuario, is_public = false y una columna adicional is_ai_generated = true para distinguirlas de las recetas creadas manualmente.

EDICIÓN ANTES DE GUARDAR

Las recetas IA guardadas se pueden editar completamente antes de guardar: cambiar nombre, ajustar cantidades de ingredientes con sliders, añadir o quitar ingredientes, y modificar los pasos. El usuario nunca está forzado a guardar la receta exactamente como la generó la IA.

```
// Migración de base de datos para recetas generadas por IA

-- Columna adicional en la tabla recipes para recetas IA

ALTER TABLE public.recipes

  ADD COLUMN is_ai_generated BOOLEAN DEFAULT false,

  ADD COLUMN ai_context JSONB,          -- contexto que generó la receta (ingrediente requerido, modo, macros)

  ADD COLUMN ai_model TEXT;             -- 'gemini-1.5-flash' | 'gpt-4o-mini'

-- Ejemplo de ai_context almacenado:
-- {
--   "required_ingredients": ["zanahoria"],
--   "mode": "protagonist",
--   "pending_macros": { "kcal": 600, "protein_g": 45 },
--   "generated_at": "2026-05-22T18:30:00Z"
-- }
```

15.8 Integración con el módulo de nutrición

Esta es la integración más importante del módulo: cuando el usuario busca recetas con un ingrediente concreto, si el módulo de nutrición está activo, los resultados se ordenan y ajustan automáticamente en base a los macros pendientes del día.

Situación del usuario	Comportamiento de FoodOS
Lleva 800 kcal a las 15h con objetivo de 2.700 kcal al día	Las recetas se ordenan priorizando las de mayor densidad calórica que usen zanahoria. El módulo de nutrición está activo.
Le faltan 60g de proteína para el objetivo diario	Las recetas con zanahoria que también incluyan fuente proteica (pollo, huevos, legumbres) se ordenan por prioridad.
Está en modo ahorro máximo (presupuesto con límite de €1,50 por ración)	Sólo se muestran recetas con zanahoria cuyo coste estimado por ración sea menor de €1,50. El módulo de nutrición está activo.
Tiene zanahoria que caduca mañana	Las recetas que usan zanahoria como ingrediente principal reciben un bonus de puntuación y se ordenan por prioridad.

15.9 Integración con el carrito de compra

Desde los resultados de búsqueda, el usuario puede añadir al carrito los ingredientes faltantes de una receta concreta sin necesidad de entrar al detalle de la receta.

- Botón 'Añadir faltantes al carrito' debajo de cada receta que tiene ingredientes en rojo.
- Solo se añaden los ingredientes que el usuario no tiene — los verdes no se añaden.
- Si el mismo ingrediente falta en varias recetas de los resultados, al añadirlo al carrito se suma la cantidad total necesaria para todas (comportamiento idéntico al carrito multi-receta).
- El carrito muestra de qué receta viene cada ingrediente: 'caldo vegetal — para Crema de zanahoria'.

15.10 Modelo de datos — tabla de búsquedas y caché

Para mejorar la experiencia y evitar llamadas innecesarias a la IA, FoodOS cachea las recetas generadas recientemente y almacena un historial de búsquedas del usuario.

```
// Tablas de soporte para búsqueda y caché de IA

-- Historial de búsquedas por ingrediente (para sugerencias futuras)

CREATE TABLE public.ingredient_searches (

  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),

  user_id           UUID NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,

  required_ingredients TEXT[] NOT NULL,

  excluded_ingredients TEXT[] DEFAULT '{}',

  mode              TEXT NOT NULL,

  result_count      INTEGER,

  ai_generated       BOOLEAN DEFAULT false,

  searched_at       TIMESTAMPTZ DEFAULT now()

);

-- Caché de recetas IA generadas (evita regenerar la misma receta)

-- Clave de caché: hash(required_ingredients + mode + inventario_hash)

CREATE TABLE public.ai_recipe_cache (

  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),

  cache_key          TEXT UNIQUE NOT NULL,      -- SHA256 del contexto

  recipes_json       JSONB NOT NULL,            -- array de recetas generadas

  ai_model           TEXT DEFAULT 'gemini-1.5-flash',

  created_at         TIMESTAMPTZ DEFAULT now(),

  expires_at         TIMESTAMPTZ DEFAULT now() + INTERVAL '6 hours'

  -- Las recetas IA caducan: el inventario cambia, los macros cambian

);

-- Índices
```



```
CREATE INDEX idx_ingredient_searches_user ON public.ingredient_searches(user_id, searched_at DESC);

CREATE INDEX idx_ai_cache_key ON public.ai_recipe_cache(cache_key);

CREATE INDEX idx_ai_cache_expires ON public.ai_recipe_cache(expires_at);

-- Limpiar caché expirado (ejecutar como cron job en Supabase Edge Functions)

CREATE OR REPLACE FUNCTION cleanup_expired_ai_cache()

RETURNS void AS $$

    DELETE FROM public.ai_recipe_cache WHERE expires_at < now();

$$ LANGUAGE sql;
```

15.11 Estimación de consumo de IA para este módulo

Acción del usuario	Requests a Gemini	Tokens estimados	Coste (Gemini gratis)
Búsqueda modos 1, 2 o 3 (sin IA)	0	0	€0 — sin IA
Generar recetas con IA (modo 4)	1 req por búsqueda	~2.000–3.500 tokens	€0 (dentro de cuota)
Guardar receta generada	0	0	€0 — solo DB
Re-generar con otros ingredientes	1 req adicional	~2.000–3.500 tokens	€0 (dentro de cuota)
Usuario muy activo (5 generaciones/día)	5 req/día	~15.000 tokens/día	€0 (<<1.500 req/día)

EFICIENCIA DE CONSUMO IA

El modo de generación IA solo se ejecuta cuando el usuario lo activa explícitamente (modo 4). Los otros tres modos son búsquedas en base de datos pura — cero llamadas a la IA, cero coste. Esto significa que la gran mayoría del uso del módulo no consume cuota de Gemini.

15.12 Resumen de pantallas del módulo

Pantalla	Descripción	Componentes clave
Búsqueda principal	Campo de texto + chips de ingredientes + selector de modos	SearchBar, IngredientChips, ModeSelector, RecipeResultList
Resultados con semáforo	Cards de recetas con indicadores de disponibilidad	RecipeCard, IngredientBadge, AvailabilityBadge, AddToCartButton
Receta IA generada	Card especial con badge 'IA · generada para ti', ingredientes y botón de regenerar	AIRecipeCard, GeneratedBadge, Ingredients, RegenerateButton
Edición pre-guardado	Formulario editable con nombre, ingredientes ajustables y botón de guardar	RecipeEditorForm, IngredientsList, SaveButton
Detalle de receta guardada	Vista completa de la receta generada, ya en la colección de recetas	RecipeDetail (componente existente, sin cambios)

Landing Page — Página de Presentación de FoodOS

FoodOS incluye una landing page pública que actúa como punto de entrada para nuevos usuarios antes de registrarse. Esta página explica la propuesta de valor de la app, muestra sus funcionalidades principales y ofrece las tres vías de descarga (iOS/Android, web y Windows), además del formulario de inicio de sesión.

16.1 Propósito y audiencia

La landing es la primera impresión de FoodOS para cualquier persona que llegue a través de un enlace compartido, una búsqueda o una recomendación. Su objetivo principal no es vender, sino comunicar con claridad qué hace la app en menos de 10 segundos y ofrecer la ruta de descarga o registro más corta posible.

16.2 Estructura y secciones

Sección	Contenido	Altura scroll	Animación principal
Hero scrubbing	Vídeo controlado por scroll + título + CTA	500vh	GSAP ScrollTrigger scrub
Ticker	Cinta de características en bucle infinito	auto	CSS animation infinite
Features bento	6 tarjetas en grid asimétrico 12 columnas	auto	GSAP reveal + Anime.js hover
Cómo funciona	4 pasos con línea conectora	auto	Anime.js stagger
Descargar	QR móvil + web + Windows exe	auto	Anime.js QR build from center
Iniciar sesión	Google OAuth + email	auto	GSAP fade-in
Footer	Links + copyright	auto	—

16.3 El efecto estrella — Video Scrubbing

El video scrubbing es la técnica que hace que el vídeo de fondo no se reproduzca automáticamente, sino que sea el usuario con su scroll quien controla en qué fotograma del vídeo se encuentra. Apple lo usa en todas sus páginas de producto. Para FoodOS, el vídeo de frutas cayendo de una caja se convierte en el protagonista visual del hero — las frutas caen al bajar y suben al subir.

TÉCNICA: VIDEO SCRUBBING

Cómo funciona técnicamente: la sección del hero tiene una altura de 500vh (cinco veces la pantalla), lo que da margen de scroll suficiente para que el usuario recorra todo el vídeo sin prisas. El contenedor interior es position: sticky, así permanece fijo en pantalla mientras el resto de la página sube. GSAP ScrollTrigger mapea el progreso del scroll (0→1) al currentTime del vídeo (0→duration), actualizando fotograma a fotograma en tiempo real.

```
// video-scrubbing.js

// Implementación del video scrubbing con GSAP ScrollTrigger
```

```
const video = document.getElementById('hero-video');

video.addEventListener('loadedmetadata', () => {

  ScrollTrigger.create({

    trigger: '#scrub-section',    // sección de 500vh de altura

    start: 'top top',

    end: 'bottom bottom',

    scrub: true,                // vincula directamente al scroll sin lerp

    onUpdate: (self) => {

      // self.progress: 0 (inicio) → 1 (final del scroll)

      video.currentTime = self.progress * video.duration;

      // Barra de progreso visual

      progressBar.style.width = (self.progress * 100) + '%';

      // El texto del hero se desvanece a medida que scrolleas

      const alpha = Math.max(0, 1 - self.progress * 3.2);

      gsap.set('.hero-content', { opacity: alpha });

    }

  });

});

// El vídeo DEBE tener: muted, playsinline, preload="auto"

// Sin estos atributos el navegador bloquea la manipulación de currentTime
```

16.4 Stack técnico de la landing

Tecnología	Versión	Uso en la landing	CDN / npm
GSAP	3.12.5	ScrollTrigger scrub, reveal, parallax	cdnjs.cloudflare.com
Anime.js	3.2.2	Stagger steps, hover icons, QR build	cdnjs.cloudflare.com
DM Serif Display	Variable	Tipografía display para títulos	Google Fonts
Outfit	Variable	Tipografía de cuerpo y UI	Google Fonts
DM Mono	Variable	Tipografía monoespaciada para labels	Google Fonts
CSS nativo	—	Grid, sticky, animaciones ticker, noise	—
Vanilla JS	ES2020+	Lógica de scroll, build QR, nav state	—

16.5 Gestión de rutas tras el login

Cuando el usuario inicia sesión desde la landing, el flujo de redirección depende de la plataforma:

- **Web:** el botón de Google OAuth llama a Supabase Auth, que redirige a /dashboard tras la autenticación. La landing y el dashboard son dos rutas de Next.js distintas.
- **App móvil:** el QR lleva a la App Store o Google Play. Tras instalar, el primer onboarding aparece directamente en la app.
- **Windows:** el instalador .exe descarga la app Electron/Tauri que usa la misma sesión de Supabase Auth.
- **PWA:** el botón 'Instalar como PWA' añade la web app al escritorio/home screen del dispositivo.

PRODUCCIÓN: NEXT.JS + CLOUDINARY

La landing está diseñada como un archivo HTML standalone durante el desarrollo (sin servidor necesario) pero en producción vivirá en Next.js como la ruta raíz '/' con SSG (Static Site Generation) para máximo rendimiento en Google PageSpeed y SEO. El vídeo se servirá desde Cloudinary con transformaciones automáticas de calidad adaptativa según la conexión del usuario.

17

GSAP — GreenSock Animation Platform

GSAP v3.12.5 — MIT License

<https://gsap.com> · npm: `gsap` · CDN: `cdnjs.cloudflare.com/ajax/libs/gsap/`

La librería de animación JavaScript más utilizada en producción. Soporta animaciones de propiedades CSS, SVG, Canvas, Three.js y WebGL. Compatible con React, Vue, Next.js y Vanilla JS. El plugin ScrollTrigger es el estándar de facto para animaciones vinculadas al scroll.

17.1 Por qué GSAP para FoodOS

CSS animations son suficientes para transiciones simples, pero FoodOS necesita animaciones que respondan al scroll, que se encadenen en secuencias precisas y que trabajen en todos los navegadores de forma idéntica. GSAP resuelve los tres problemas con una API coherente y un rendimiento superior al navegador nativo en la mayoría de los casos.

17.2 Plugins de GSAP utilizados en FoodOS

Plugin	Gratis/Pro	Uso en FoodOS	Importación
ScrollTrigger	Gratis	Video scrubbing, reveal cards, parallax, nav state	gsap/ScrollTrigger
gsap core	Gratis	Timelines hero, tweens individuales, to/from/set	gsap
Flip	Gratis	Fase 2: transición de tarjetas de inventario al detalle	gsap/Flip
DrawSVG	Club GSAP	Fase 3: animación de anillo de macros SVG	gsap/DrawSVGPlugin
MorphSVG	Club GSAP	Fase 3: transición entre iconos de categoría	gsap/MorphSVGPlugin
ScrollSmoother	Club GSAP	Opcional: scroll suavizado en la landing	gsap/ScrollSmoother

COSTE DE GSAP

Los plugins marcados como 'Club GSAP' requieren suscripción (~\$150/año). NO son necesarios para la versión inicial de FoodOS — todo lo esencial (ScrollTrigger, timelines, tweens) es completamente gratuito. Club GSAP solo tiene sentido para efectos avanzados en versiones futuras.

17.3 Casos de uso concretos en FoodOS

Landing — Video scrubbing

```
// El núcleo del efecto de vídeo controlado por scroll

gsap.registerPlugin(ScrollTrigger);

ScrollTrigger.create({
```

```
trigger: '#scrub-section',

start: 'top top',

end: 'bottom bottom',

scrub: true,          // sin lerp: 1:1 con el scroll

onUpdate: (self) => {

    video.currentTime = self.progress * video.duration;

}

});
```

Landing — Hero entrance timeline

```
// Secuencia de entrada del hero con stagger

const tl = gsap.timeline({ delay: 0.15 });

tl.to('#badge',    { opacity: 1, y: 0, duration: 0.6, ease: 'power3.out' })

    .to('#title',  { opacity: 1, y: 0, duration: 0.9, ease: 'power4.out' }, '-=0.3')

    .to('#subtitle', { opacity: 1, y: 0, duration: 0.7, ease: 'power3.out' }, '-=0.5')

    .to('#ctas',    { opacity: 1, y: 0, duration: 0.6, ease: 'power3.out' }, '-=0.4');
```

App — Reveal de tarjetas de inventario al hacer scroll

```
// Tarjetas de inventario que aparecen al entrar en viewport

gsap.utils.toArray('.inventory-card').forEach((card, i) => {

    gsap.fromTo(card,

        { opacity: 0, y: 32, scale: 0.97 },

        {

            opacity: 1, y: 0, scale: 1,

            duration: 0.65, ease: 'power3.out',

            delay: (i % 3) * 0.08,    // stagger por columna en grid de 3

            scrollTrigger: {

                trigger: card,

                start: 'top 88%',

                once: true              // solo se anima la primera vez

            }

        }

    );

});
```

App — Transición de pantalla (Flip plugin)

```
// Al pulsar una tarjeta de receta, la tarjeta 'vuela'
// y se convierte en la pantalla de detalle

import { Flip } from 'gsap/Flip';

gsap.registerPlugin(Flip);

function openRecipeDetail(card) {

  const state = Flip.getState(card); // captura posición actual
  card.classList.add('is-detail'); // mueve al nuevo contexto DOM

  Flip.from(state, {

    duration: 0.5,

    ease: 'power2.inOut',

    nested: true,

    onComplete: () => console.log('Transición completa')

  });
}
```

App — Parallax en el header del dashboard

```
// El fondo del header se mueve más lento que el contenido

gsap.to('.dashboard-bg', {

  yPercent: -20,

  ease: 'none',

  scrollTrigger: {

    trigger: '.dashboard',

    start: 'top top',

    end: 'bottom top',

    scrub: true

  }

});
```

17.4 Instalación y configuración en Next.js

```
// Instalación GSAP en Next.js 14 App Router

# Instalación

npm install gsap

# En Next.js con App Router – registrar plugins en el cliente

// app/providers/gsap-provider.tsx
```

```
'use client';

import { useEffect } from 'react';
import gsap from 'gsap';
import { ScrollTrigger } from 'gsap/ScrollTrigger';
import { Flip } from 'gsap/Flip';

export function GSAPProvider({ children }: { children: React.ReactNode }) {
  useEffect(() => {
    gsap.registerPlugin(ScrollTrigger, Flip);

    // Importante en Next.js: refresh ScrollTrigger en route changes
    return () => { ScrollTrigger.getAll().forEach(t => t.kill()); };
  }, []);

  return <>{children}</>;
}

// En app/layout.tsx
import { GSAPProvider } from '@providers/gsap-provider';

export default function RootLayout({ children }) {
  return <html><body><GSAPProvider>{children}</GSAPProvider></body></html>;
}
```

17.5 Rendimiento y buenas prácticas

- **will-change: transform** en elementos que van a animarse con GSAP para que el navegador los promueva a su propia capa GPU.
- **once: true** en ScrollTrigger para animaciones de reveal — así no recalcula en cada scroll.
- **gsap.set()** en vez de CSS para el estado inicial — evita el flash of unstyled content (FOUC).
- **ScrollTrigger.refresh()** después de cambios de layout dinámico (imágenes que cargan, acordeones, etc.).
- **context()** y **revert()** en React para limpiar animaciones al desmontar componentes y evitar memory leaks.
- Usar **gsap.matchMedia()** para desactivar animaciones pesadas en móvil (prefers-reduced-motion).

18

Anime.js — JavaScript Animation Engine

Anime.js v3.2.2 — MIT License

https://animejs.com · npm: animejs · GitHub: 4.9k líneas de código

Librería de animación ligera (~17KB minificada) con una API muy concisa. Ideal para animaciones de UI, staggering, keyframes y animaciones matemáticas complejas. Complementa a GSAP — se usa para micro-interacciones y efectos de entrada de componentes donde GSAP sería excesivo.

18.1 GSAP vs Anime.js — cuándo usar cada uno en FoodOS

Ambas librerías coexisten en FoodOS con responsabilidades diferenciadas. No compiten, se complementan:

Criterio	GSAP	Anime.js
Scroll animations	✓ ScrollTrigger (nativo)	✗ No tiene
Timelines complejas	✓ Con labels, callbacks, seek()	✓ Pero más limitado
Stagger de elementos	✓ gsap.utils.toArray + stagger	✓ anime.stagger() — más conciso
SVG path morph	✓ MorphSVG (plugin Club)	✓ Nativo con path morph
Keyframes	✓ Sintaxis array	✓ Sintaxis array más limpia
Peso (bundle)	~27KB min+gzip (core)	~6KB min+gzip
Counters animados	✓ Con ticker manual	✓ Nativo con { val: N }
Grids stagger	✓ gsap.utils.distribute()	✓ anime.stagger({ grid })
Uso en FoodOS	Scroll, timelines hero, transiciones	Micro-interacciones, contadores, QR

18.2 Casos de uso concretos en FoodOS

Landing — Pasos 'Cómo funciona' con stagger

```
// Los 4 pasos aparecen en cascada al entrar en viewport
// Disparado por ScrollTrigger de GSAP como puente
ScrollTrigger.create({
  trigger: '.how-steps',
  start: 'top 76%',
  once: true,
  onEnter: () => {
    anime({
      targets: '.step-card',
```

```
    opacity: [0, 1],

    translateY: [24, 0],

    delay: anime.stagger(110),    // 110ms entre cada paso

    duration: 680,

    easing: 'easeOutExpo'

  });

}

});
```

Landing — QR que se construye desde el centro

```
// Las celdas oscuras del QR aparecen desde el centro hacia fuera

// Crea un efecto de 'scan' o 'materialización' muy visual

anime({

  targets: '.qr-cell.dark',

  opacity: [0, 1],

  scale: [0.3, 1],

  delay: anime.stagger(18, {

    grid: [7, 7],          // grid de 7x7 celdas

    from: 'center'         // empieza desde el centro

  }),

  duration: 450,

  easing: 'easeOutBack'

});
```

App — Contador animado de calorías diarias

```
// El número de calorías se cuenta desde 0 hasta el valor real

// Usado en el anillo de progreso del dashboard diario

function animateCalories(targetKcal: number) {

  anime({

    targets: { val: 0 },

    val: targetKcal,

    round: 1,

    duration: 1800,

    easing: 'easeOutExpo',

    update: function(anim) {

      const current = anim.animations[0].currentValue;
```

```
document.getElementById('kcal-display').textContent =  
    Math.round(current).toLocaleString('es');  
  
    }  
});  
  
}  
  
// Ejemplo: animateCalories(1805); → cuenta de 0 a 1.805
```

App — Micro-interacción en icono de categoría financiera

```
// Al pulsar una categoría de gasto, el icono rebota  
  
// Da feedback táctil visual sin ser intrusivo  
  
function animateCategorySelect(iconEl: HTMLElement) {  
  
    anime({  
  
        targets: iconEl,  
  
        scale: [1, 1.25, 0.9, 1.05, 1], // keyframes de rebote  
  
        rotate: [0, -8, 6, -3, 0],  
  
        duration: 480,  
  
        easing: 'easeInOutQuad'  
  
    });  
  
}
```

App — Tarjeta de alimento deslizándose fuera al consumir

```
// Al marcar un alimento como consumido, se desliza y desaparece  
  
function removeInventoryItem(card: HTMLElement, onComplete: () => void) {  
  
    anime({  
  
        targets: card,  
  
        translateX: [0, '110%'], // se va a la derecha  
  
        opacity: [1, 0],  
  
        height: [card.offsetHeight, 0],  
  
        marginBottom: [8, 0],  
  
        duration: 380,  
  
        easing: 'easeInCubic',  
  
        complete: onComplete // eliminar del DOM al terminar  
  
    });  
  
}
```

App — Anillo de progreso SVG de macros

```
// Los arcos SVG del anillo se dibujan animados al cargar

// Simula un 'llenado' de los macros del día

function animateMacroRing(proteinPct: number, carbsPct: number) {

  const circumference = 2 * Math.PI * 52; // radio 52px

  anime({

    targets: '#protein-arc',

    strokeDasharray: [

      `0 ${circumference}`,

      `${circumference * proteinPct / 100} ${circumference}`

    ],

    duration: 1200,

    easing: 'easeOutQuart',

    delay: 200

  });

  anime({

    targets: '#carbs-arc',

    strokeDasharray: [

      `0 ${circumference}`,

      `${circumference * carbsPct / 100} ${circumference}`

    ],

    duration: 1200,

    easing: 'easeOutQuart',

    delay: 400

  });

}
```

18.3 Instalación en Next.js con Expo compartiendo lógica

```
// Estrategia cross-platform: Anime.js (web) + Reanimated (móvil)

# Web (Next.js)

npm install animejs

npm install --save-dev @types/animejs

# Expo/React Native: Anime.js NO funciona directamente en RN

# (manipula DOM que no existe en móvil).

# En móvil usar React Native Reanimated o Moti en su lugar:
```

```
npx expo install react-native-reanimated moti

# packages/utils/animations.ts – Abstracción compartida

// En web: usa anime.js

// En móvil: usa Reanimated/Moti

// El componente no sabe qué librería usa por debajo

export const platformAnimations = {

  staggerFadeIn: (targets: string, delay: number) => {

    if (typeof window !== 'undefined') {

      // Web: Anime.js

      return anime({ targets, opacity: [0,1], translateY: [20,0],

        delay: anime.stagger(delay), duration: 600, easing: 'easeOutExpo' });

    }

    // Móvil: se maneja en el componente nativo con Moti

  }

};
```

NOTA: ANIME.JS v4 EN DESARROLLO

Anime.js v4 está en desarrollo activo (actualmente en beta). La API cambia significativamente respecto a v3. Para FoodOS se usa v3.2.2 que es la versión estable. Cuando v4 salga de beta se puede migrar ya que el cambio principal es sintáctico, no conceptual.

19

21st.dev Magic MCP — Generación de UI por IA en el IDE

21st.dev Magic MCP Beta — MIT License

<https://21st.dev/mcp> · [npm: @21st-dev/magic](#) · [GitHub: 21st-dev/magic-mcp](#) (4.4k)

Servidor MCP (Model Context Protocol) que permite generar componentes React/TypeScript de calidad profesional directamente desde el chat del IDE usando lenguaje natural. Funciona con Cursor, Windsurf, VS Code + Cline y Claude Code. Backed by Y Combinator.

19.1 Qué es un MCP y cómo encaja con FoodOS

MCP (Model Context Protocol) es un estándar abierto de Anthropic que permite a los modelos de IA conectarse con herramientas y servicios externos. El MCP de 21st.dev conecta el agente de IA de tu IDE con la librería de componentes de 21st.dev, generando código React real y escribiéndolo directamente en los archivos del proyecto.

19.2 Las tres herramientas del MCP

Herramienta	Coste	Qué hace	Cuándo usarlo en FoodOS
Inspiration Search	Gratis	Búsqueda semántica entre miles de componentes de 21st.de	Al momento de crear cualquier componente
SVG Icon Search	Gratis	Busca logos e iconos SVG en svgl.app (Google, Apple, Supabase, (Google etc)), los iconos de	Al momento de crear cualquier componente
Magic Generate	20\$/mes Pro	Genera 5 variantes de un componente, las muestra en el navegador	Al momento de crear cualquier componente

19.3 Instalación — un solo comando

[illegible]

```
"mcpServers": {  
  "@21st-dev/magic": {  
    "command": "npx",  
    "args": ["-y", "@21st-dev/magic@latest", "API_KEY=\"tu-api-key\""]  
  }  
}  
  
# Obtener la API key en: https://21st.dev/magic/console
```

19.4 Comandos y ejemplos de uso para FoodOS

Una vez instalado el MCP, desde el chat del agente en tu IDE puedes escribir comandos en lenguaje natural. Estos son los comandos concretos que usarás al construir FoodOS:

Comando:

```
/ui /ui tarjeta de producto del inventario con emoji de alimento, nombre, cantidad, unidad, fecha de caducidad con badge de color (verde/amarillo/rojo), y botón de consumir. Fondo oscuro #0f130c, acento verde #4ade80.
```

→ Genera la InventoryCard del módulo de almacén

Comando:

```
/ui /ui anillo de progreso de calorías diarias con tres barras de macros al lado (proteína azul, carbos naranja, grasas morado). El anillo tiene el porcentaje en el centro con la cifra grande. Estilo dark, tipografía monospace para los números.
```

→ Genera el DailyMacroRing del dashboard

Comando:

```
/ui /ui formulario de añadir gasto con: toggle ingreso/gasto, input de importe grande, descripción, fecha, grid de 9 categorías con emojis seleccionables (activa borde verde), y selector de frecuencia (una vez / mensual / anual). Dark mode.
```

→ Genera el AddExpenseForm del módulo financiero

Comando:

```
/ui /ui tarjeta de receta para feed social con: imagen de fondo, título en overlay, autor con avatar, macros en fila (kcal, proteína, carbos), chips de ingredientes con semáforo verde/amarillo/rojo, y botones de like, guardar y añadir al carrito.
```

→ Genera la RecipeCard del feed social

Comando:

```
/ui /ui pantalla de perfil nutricional con 4 pasos: datos físicos, objetivo (4 cards seleccionables), nivel de actividad (5 opciones con slider), y resumen calculado. Indicador de progreso en la parte superior.
```

→ Genera el NutritionOnboarding de 4 pasos

19.5 Flujo de trabajo recomendado con 21st.dev + FoodOS

Paso	Acción	Herramienta	Resultado
1	Antes de crear un componente nuevo, buscar inspiración de diseño (gratis)	Inspiration Search (gratis)	Referencia de código y estilos
2	Si hay login o logos de marcas, buscar el SVG limpio	SVG Search (gratis)	SVG limpio insertado en el código
3	Describir el componente con /ui y generar 5 variantes	Magic Generate (Pro)	5 opciones React/TSX en el navegador
4	Elegir la variante que más se acerca al diseño de FoodOS en el navegador	Posta de FoodOS	Código enviado al IDE automáticamente
5	Ajustar colores, tipografía y lógica al design system de FoodOS	Style Guide de FoodOS	Componente final integrado en el proyecto
6	Si el resultado no es ideal, usar 'Chat to Edit' para generar otra variante	Magic Generate Pro	Variante mejorada sin salir del IDE

19.6 Limitaciones y cuándo NO usarlo

- **HTML puro:** el MCP genera React/TypeScript. No sirve para el HTML standalone de la landing (GSAP + Anime.js). Usar solo cuando el proyecto esté en Next.js.
- **Lógica de negocio compleja:** el MCP genera UI, no lógica. El cálculo de TMB/TDEE, el scrubbing del vídeo o las llamadas a Supabase se escriben manualmente.
- **Sin contexto de tu proyecto:** el agente no sabe automáticamente qué variables CSS usa FoodOS. Hay que indicarlo en el prompt ('/ui ... acento verde #4ade80, fondo #0f130c').
- **Componentes muy específicos:** para cosas muy concretas de FoodOS (el semáforo de caducidad o el chip de ingrediente obligatorio) es más rápido escribirlo a mano que describírselo al MCP.

RECOMENDACIÓN FINAL PARA FOODOS

RESUMEN PARA FOODOS: instalar el MCP de 21st.dev en Cursor desde el primer día. Usar Inspiration Search (gratis) siempre antes de crear un componente. Usar Magic Generate (Pro, 20\$/mes) para los componentes más visuales y complejos como las tarjetas del dashboard, el anillo de macros y el formulario de finanzas. El tiempo ahorrado en esos componentes clave justifica el coste del plan Pro en las primeras semanas de desarrollo.

20

Resumen — Ecosistema de Librerías de FoodOS

Visión consolidada de todas las librerías de UI, animación y herramientas de desarrollo que componen el stack de FoodOS, con su función exacta, coste y en qué parte de la app se usan.

20.1 Librerías de animación

Librería	Versión	Plataforma	Uso principal en FoodOS	Coste
GSAP (core)	3.12.5	Web	Timelines hero, reveal cards, parallax dashboard	Gratis
GSAP ScrollTrigger	3.12.5	Web	Video scrubbing, trigger de secciones	Gratis
GSAP Flip	3.12.5	Web	Transición tarjeta → detalle receta	Gratis
Anime.js	3.2.2	Web	Stagger steps, counters, QR build, micro-hover	Gratis
Reanimated	3.x	iOS/Android	Animaciones nativas en la app móvil	Gratis
Moti	0.x	iOS/Android	Componentes animados declarativos en Expo	Gratis
Framer Motion	11.x	Web (opcional)	Alternativa a GSAP para componentes React	Gratis

20.2 Herramientas de desarrollo y generación de UI

Herramienta	Tipo	Uso en FoodOS	Coste
21st.dev MCP	MCP Server	Generación de componentes React desde el IDE	Gratis + Pro 20\$/mes
Cursor	IDE	Editor principal con agente IA integrado	Gratis + Pro 20\$/mes
Claude Code	CLI agent	Automatización de tareas, refactoring, tests	Por tokens (API)
v0 by Vercel	Web app	Alternativa a 21st.dev para generar UI	Gratis + Pro
Storybook	Dev tool	Documentación y testeo visual de componentes	Gratis
Tailwind CSS	Framework	Estilos utilitarios en la web de FoodOS	Gratis (OSS)

20.3 Tabla de decisión — qué usar según el caso

Necesitas...	Usa...	Por qué
Animar al hacer scroll	GSAP ScrollTrigger	Es el estándar. Sin alternativa real en web
Efecto de vídeo scrubbing	GSAP ScrollTrigger	scrollTrigger.scrub + video.currentTime
Stagger de elementos en lista	Anime.js	anime.stagger() más conciso que gsap
Contadores de números animados	Anime.js	Nativo con { val: N }, más simple
Transición entre pantallas en web	GSAP Flip	Flip captura y anima cambios de layout
Animaciones en la app móvil (Expo)	Reanimated + Moti	Únicas que funcionan en React Native
Crear componente React nuevo rápido	21st.dev Magic MCP	5 variantes en segundos desde el IDE

Necesitas...	Usa...	Por qué
Buscar icono de marca (Google, Apple...)	21st.dev SVG Search	Gratis, directo en el código
Animación simple de un botón/hover	CSS transition	No necesitas JS para eso
Entrada de página / hero	GSAP timeline	Control total de la secuencia

21 Design System — Identidad Visual de FoodOS

El design system de FoodOS define la identidad visual completa de la aplicación: paleta de colores, tipografía, espaciado, iconografía, tono de las animaciones y directrices del logo. Cualquier nueva pantalla, asset o componente debe seguir estas reglas para mantener coherencia visual en todas las plataformas.

21.1 Concepto visual general

FoodOS tiene una estética que llamamos **Organic Dark**: oscura como una cocina profesional de noche, con acentos verdes vivos que evocan frescura y naturaleza. No es el dark mode genérico de las apps de finanzas — es más cálido, más orgánico, con texturas sutiles de ruido y tipografía serif en los titulares que recuerda a una carta de restaurante de lujo contemporáneo.

Dimensión	Elección	Por qué
Fondo base	Negro orgánico #070a05	Más cálido que negro puro, evoca cocina/naturaleza nocturna
Acento primario	Verde lima #4ade80	Frescura, salud, energía — diferencia de apps de fitness azules
Acento secundario	Ámbar #fb923c	Urgencia suave (caducidades, alertas), cálido y apetecible
Tipografía display	DM Serif Display (serif)	Elegancia inesperada, contraste con lo digital, carta de menú
Tipografía body	Outfit (geometric sans)	Legible, moderno, tech sin ser frío
Tipografía mono	DM Mono	Para datos, labels, código, macros numéricos
Textura	Noise overlay CSS	Añade profundidad orgánica, evita aspecto plástico
Tono de voz	Directo, cálido, sin jerga	Como un amigo que sabe de nutrición y finanzas

21.2 Paleta de colores completa

Colores base (fondos y superficies):

BG Base

BG Layer 2

BG Layer 3

Surface

Surface 2

Surface 3

#070a05

#0d120a

#131a0f

#182013

#1e2a18

#212918

Colores de acento y semánticos:

Green

Green Dim

Green Dark

Amber

Red

Blue

#4ade80

#22c55e

#15803d

#fb923c

#f87171

#60a5fa

Colores de texto y bordes:

Text

Muted

Hint

Border

Border 2

Border A

#f0f4ee

#7d8f78

#3d4f38

#182013

#0d120a

#4ade80

Semántica de color en la app:

Color	Hex	Uso semántico en FoodOS
Verde	#4ade80	Disponible en inventario · OK · dentro de presupuesto · objetivo cumplido · botones primarios
Ámbar	#fb923c	Caduca pronto (1-3 días) · stock bajo · presupuesto al 80% · advertencia no crítica
Rojo	#f87171	Caducado o agotado · presupuesto superado · macro no alcanzado · error
Azul	#60a5fa	Información · finanzas · datos bancarios · links · congelador
Morado	#c084fc	IA y generación · recomendaciones · insights inteligentes

21.3 Tipografía

Familia	Peso	Tamaño	Uso
DM Serif Display	Regular, Italic	clamp(34-116px)	Títulos hero, section titles, nombres de recetas, logotipo
Outfit	300/400/500/600/700/900	9-18px	Body text, labels, botones, UI en general
DM Mono	400/500	8-13px	Macros numéricos, precios, código, tags técnicos, badges

- Los títulos de sección siempre van en **DM Serif Display**. Nunca uses Outfit para un H1 principal.
- Los números importantes (calorías, euros, gramos) siempre en **DM Mono** — da sensación de precisión técnica.
- **Italic en verde** (#4ade80) para las palabras de énfasis dentro de los títulos.
- **Letter-spacing de 1.5-2.5px** en labels de categoría (FONT-SIZE 10-11px, UPPERCASE).
- **Line-height 0.92-0.95** para títulos display grandes — permite esa compresión que les da fuerza visual.

21.4 Espaciado y bordes

Token	Valor	Uso
--radius-sm	8-10px	Chips, badges, inputs pequeños
--radius-md	12-14px	Botones, tarjetas pequeñas
--radius-lg	18-20px	Cards principales, modales
--radius-xl	24-28px	Hero sections, full-bleed cards
--radius-full	9999px	Pills, avatares, toggles
Border width	0.5-1px	Bordes de cards (nunca más de 1.5px excepto activos)
Gap card grid	10-12px	Gap entre cards en bento grid
Section padding	80-120px	Padding vertical de cada sección
Noise opacity	0.03-0.05	La textura de ruido del fondo — muy sutil

21.5 Animaciones — principios y reglas

Las animaciones de FoodOS deben sentirse **orgánicas y con peso**, no mecánicas ni bouncy de forma exagerada. La física que inspira el movimiento es la de los alimentos: suaves, con masa, naturales.

Tipo de movimiento	Easing recomendado	Duración típica	Librería
Entrada de página / hero	power4.out / expo.out	700-1100ms	GSAP timeline
Reveal al hacer scroll	power3.out	600-750ms	GSAP ScrollTrigger
Stagger de lista	easeOutExpo	600ms c/ 80-120ms delay	Anime.js
Hover en card	ease-out	200-250ms	CSS transition
Micro-interacción icono	easeOutBack	260-320ms	Anime.js
Contador numérico	easeOutExpo	1500-2000ms	Anime.js
Modal entrada	power3.out + scale	350ms	GSAP / CSS
Notificación toast	easeOutBack → easeInBack	300ms / 250ms	CSS + JS
Personaje mascota idle	easeInOutSine loop	3000-4000ms	Anime.js loop

ACCESIBILIDAD — OBLIGATORIO

REGLA ABSOLUTA: respeta siempre prefers-reduced-motion. Envuelve todas las animaciones en gsap.matchMedia() o @media (prefers-reduced-motion: reduce) para usuarios con sensibilidad al movimiento. La app debe ser completamente usable sin ninguna animación.

21.6 El logo de FoodOS

El logo sigue un concepto tipográfico minimalista. No hay un icono separado — el logo ES el nombre tipografiado de una forma específica. Esto lo hace infinitamente escalable y reconocible.

Elemento	Especificación	Notas
Tipografía	DM Serif Display Regular + Italic	'Food' en regular, 'OS' en regular o italic según contexto
Color 'Food'	#4ade80 (verde lima)	En fondos oscuros. En fondos claros: #2e7d32
Color 'OS'	#f0f4ee (texto principal)	En fondos oscuros. En fondos claros: #1a1a1a
Tamaño mínimo	14px en digital / 12mm en impresión	Por debajo de este tamaño usar solo el isotipo
Espaciado letras	Letter-spacing: -0.3px	Leve compresión que da elegancia
Versión oscura	Food(verde) + OS(blanco) sobre #070a05	Uso principal en la app
Versión clara	Food(verde oscuro) + OS(negro) sobre blanco	Para impresión y fondos claros
Isotipo	Letra 'F' estilizada o  emoji	Para favicon, app icon, notificaciones push
Prohibido	Estirar, rotar, cambiar fuente, añadir sombra dura	El logo no se distorsiona nunca

APP ICON

El app icon para iOS/Android es un cuadrado con fondo #070a05, las esquinas redondeadas nativas de la plataforma (no añadadas tú el redondeo), y el texto 'FoodOS' centrado en DM Serif Display en su versión completa, o solo 'F' en verde lima para tamaños de 40px o menos.

22 Prompts Base — Generación de Assets Visuales

Todos los assets visuales de FoodOS (imágenes, ilustraciones, iconos, vídeos, fondos) deben generarse usando estos prompts base. Son el ancla de consistencia visual: cualquier nueva imagen generada en el futuro, por cualquier IA, debe incluir el bloque STYLE ANCHOR completo para que el resultado pertenezca inequívocamente al universo visual de FoodOS.

22.1 Style Anchor — el bloque que nunca cambia

Este fragmento debe añadirse AL FINAL de cualquier prompt de imagen, sea lo que sea lo que estés generando:

STYLE ANCHOR — Añadir siempre al final de todos los prompts de FoodOS

```
--style-anchor--

Visual style: organic dark aesthetic, deep forest-black backgrounds (#070a05),
vibrant lime-green accents (#4ade80), warm amber highlights (#fb923c).

Lighting: soft directional natural light, subtle green rim light on subjects.

Texture: fine film grain overlay, organic imperfections welcome.

Mood: premium restaurant at night meets health-tech startup.

Color palette strictly: #070a05 bg, #4ade80 primary, #fb923c secondary,
#f0f4ee text, no other hues unless explicitly stated.

Typography (if text in image): DM Serif Display, white or lime-green only.

No gradients with purple, no neon glow, no white backgrounds unless explicitly asked.

Quality: photorealistic or semi-stylized (not cartoon unless specified),
8K, sharp focus, professional composition.
```

22.2 Prompt — Imágenes de alimentos para recetas

Usar con: Gemini Imagen 3, DALL-E 3, Midjourney, Stable Diffusion

```
Professional food photography of "[NOMBRE_RECETA]",
main ingredients visible: [INGREDIENTES_PRINCIPALES],
```

served in minimalist dark ceramic tableware,
placed on a dark slate or black marble surface,
soft natural side lighting creating gentle shadows,
shallow depth of field with creamy bokeh background,
garnished elegantly with fresh herbs,
restaurant fine-dining presentation, slightly steam rising,
color accent: a touch of green herb or lime wedge visible,
top-down or 45-degree angle, 4K, ultra-sharp.

[Pegar STYLE ANCHOR aquí]

22.3 Prompt — Imágenes de interfaz / mockups de la app

Para screenshots, promo art, capturas estilizadas

App UI mockup of FoodOS [NOMBRE_PANTALLA] screen,
shown on a [iPhone 15 Pro / Samsung Galaxy S25 / MacBook] device frame,
device floating at a slight angle (15-20 degrees tilt),
background: deep dark green-black gradient,
UI elements clearly visible with lime-green (#4ade80) accents,
soft green glow behind the screen,
premium product photography composition,
no harsh shadows, studio-quality render.

[Pegar STYLE ANCHOR aquí]

22.4 Prompt — Fondos y texturas para secciones

Para hero sections, backgrounds de landing, wallpapers

Abstract organic texture for premium food-tech app background,

[DESCRIPCIÓN: ej. 'fresh vegetables falling in slow motion'

ej. 'macro shot of green leaves with water droplets'

ej. 'dark kitchen counter with ingredients arranged'],

deep black-green color palette (#070a05 dominant),

subtle lime-green light leaks and bokeh elements,

no text, no UI elements, cinematic quality,

designed to have dark overlay for text readability,

16:9 ratio for desktop, also available in 9:16 for mobile.

[Pegar STYLE ANCHOR aquí]

22.5 Prompt — Vídeos e ilustraciones animadas

Para Veo 2 (Google), Kling, Runway, Sora

Short loop video (3-5 seconds) for FoodOS app background,

subject: [DESCRIPCIÓN: ej. 'fresh colorful fruits and vegetables falling gently

into a dark bowl in slow motion, 120fps slow-mo'],

camera: fixed position, slight depth of field,

lighting: dramatic side light with soft green rim light,

color grade: deep shadows, saturated greens and ambers,

no camera movement (or very subtle dolly),

seamless loop if possible, no audio needed,

cinematic quality, not stock-footage aesthetic.

[Pegar STYLE ANCHOR aquí]

22.6 Prompt — Iconografía y UI assets

Para Gemini Imagen 3 / DALL-E / Midjourney — iconos y elementos de UI

Flat vector icon of [NOMBRE_ICONO: ej. 'refrigerator', 'nutrition ring', 'grocery cart', 'bank connection', 'chef hat with sparkles'],

line style: clean 2px stroke, rounded caps,

color: lime-green (#4ade80) on transparent background,

size: square format, centered composition,

no gradients, no 3D effects, no shadows,

style consistent with Lucide or Tabler icon sets but organic/food-themed.

[Pegar STYLE ANCHOR aquí]

22.7 Dónde generar assets — herramientas gratuitas

Herramienta	URL	Qué genera mejor	Plan gratis
Google AI Studio	aistudio.google.com	Imágenes (Imagen 3), consistentes, fotorrealistas	Sí — con Google AI Plus
Gemini (web)	gemini.google.com	Imágenes rápidas, mockups de UI	Sí — 10 imgs/día aprox.
Adobe Firefly	firefly.adobe.com	Vectores, UI assets, texturas	25 créditos/mes gratis
Leonardo AI	app.leonardo.ai	Personajes, ilustraciones, estilos	150 tokens/día gratis
Ideogram	ideogram.ai	Texto dentro de imágenes, logos	10 imgs/día gratis
Krea AI	krea.ai	Real-time generation, fondos	Sí — limitado
Bing Image Creator	bing.com/images/create	Imágenes rápidas con DALL-E	Sí — créditos diarios
Veo 2 (Google)	labs.google (waitlist)	Vídeos cortos de alta calidad	Con Google One AI Premium

RECOMENDACIÓN DE FLUJO

FLUJO RECOMENDADO: usar Google AI Studio (con tu Google AI Plus) como herramienta principal para todo — imágenes (Imagen 3), vídeos (Veo 2) y texto (Gemini Pro). Es la opción más potente y consistente con el resto del stack de FoodOS. Para vectores e iconos, Adobe Firefly es el complemento perfecto. Siempre descargar en PNG con fondo transparente cuando sea posible.

Mascotas Personales — Tu Compañero en FoodOS

Cada usuario de FoodOS tiene UNA mascota personal que lo acompaña en toda la app. No está ligada a ningún módulo concreto — es su compañero de vida dentro de la aplicación. La elige en el onboarding y puede cambiarla cuando quiera desde Ajustes → Mi mascota. Hay 15 para elegir, todas con el mismo estilo visual pero personalidades distintas.

FILOSOFÍA — MASCOTA PERSONAL, NO ASISTENTE DE MÓDULO

CONCEPTO CLAVE: las mascotas NO son asistentes de módulo ni guías de pantalla. Son compañeros — como un Tamagotchi o una mascota de Animal Crossing. Aparecen siempre en la esquina inferior derecha, reaccionan a lo que pasa en la app (logros, alertas, inactividad) y son el 'rostro' emocional de FoodOS. El usuario las elige porque le gustan, no porque sean útiles para una tarea.

23.1 Estilo visual unificado — el mismo universo para los 15

Todos los personajes pertenecen al mismo universo visual. Un usuario que vea a cualquiera de los 15 debe reconocer inmediatamente que son de FoodOS, aunque nunca haya visto ese personaje antes. Esto requiere un sistema de reglas estrictas de diseño que todos comparten.

Regla visual	Especificación	Por qué
Proporciones chibi	Cabeza = 45-50% de la altura total del personaje	Hace los personajes más expresivos y adorables
Outline negro	Contorno negro de 2-3px, redondeado en esquinas	Une todos los estilos bajo un mismo lenguaje visual
Paleta limitada	Máximo 5-6 colores por personaje + negro + blanco	Coherencia visual, aspecto de juego indie
Ojos expresivos	Ojos grandes, circular o ovalado, alta variación	Los ojos son el 80% de la expresividad del personaje
Sin fondo	PNG transparente siempre	El personaje vive sobre cualquier pantalla de la app
Canvas cuadrado	512×512 px para generación, 128×128 en app	Proporciones fijas en toda la colección
Iluminación plana	Sin sombras volumétricas. Colores planos o con sombra 2D simple	Conservar el mismo estilo 2D flat
Acento verde lima	Cada personaje tiene AL MENOS un detalle en verde lima	Los 15 se nota visualmente con la identidad de FoodOS
Sin texto en el sprite	Nunca nombre ni letras dentro del personaje	Funciona en cualquier idioma y tamaño

REFERENCIA DE ESTILO — Cuphead meets Animal Crossing

REFERENCIA VISUAL: el estilo de FoodOS está a mitad de camino entre Cuphead (outlines negros limpios, colores vivos, personajes orgánicos) y Animal Crossing (proporciones chibi, expresiones tiernas, mundo simpático). No pixel art. No 3D. No realismo. Siempre 2D flat con outlines.

23.2 Prompt maestro de estilo — el ancla visual de los 15

Este bloque se añade al final de TODOS los prompts de generación de personajes. Nunca se modifica. Es lo que garantiza que los 15 compartan el mismo universo.

MASTER STYLE ANCHOR — añadir al final de TODOS los prompts de personajes

```
=== FOODOS MASCOT STYLE SYSTEM (DO NOT CHANGE) ===

Art style: 2D flat illustration with clean black outlines (2-3px, rounded caps).

Proportions: chibi – head is 45-50% of total body height.

Eyes: large, round or oval, highly expressive – this is the main emotional organ.

Palette: flat colors, max 5-6 colors per character + black outlines + white highlights.

Every character MUST include at least one detail in lime-green (#4ade80)

    to connect to the FoodOS brand identity.

Shading: flat color only, OR simple 2-tone shading (no gradients, no 3D render).

Outline: consistent black (or very dark color) outline around ALL shapes.

Background: fully transparent PNG.

Canvas: 512x512 pixels, character centered, leaving 10% padding on all sides.

No text, no speech bubbles, no environment background.

No realistic lighting, no 3D, no pixel art, no anime (unless character calls for it).

Consistency: the character must look like it belongs in the same universe

    as a cute food-themed 2D video game (think: Cuphead outlines + AC:NH proportions).

=== END STYLE SYSTEM ===

Output: PNG, transparent background, 512x512px, character centered.
```

23.3 Los 15 personajes — ficha completa y prompt individual

01 — ZANA

Zanahoria kawaii, femenina. Energética y motivadora.

Carrot character. Orange elongated body, chibi proportions (big head). Female, expressive big eyes with long lashes, rosy cheeks. Bright green (#4ade80) leaf hair styled upward like a ponytail. Small rounded orange arms. Cheerful open smile. Wears a tiny lime-green (#4ade80) crop top or bow detail. Front-facing idle pose, arms slightly away from body. === FOODOS MASCOT STYLE SYSTEM (see §23.2 Master Style Anchor) ===



02 — BASIL

Chef humano minimalista. Sereno y experto.

Minimalist human chef character, chibi proportions. Calm expression, half-closed confident eyes. Medium-dark skin, short neat dark hair. Dark charcoal (#1a1a1a) chef jacket, single lime-green (#4ade80) collar trim or button. Tiny chef hat (white, slightly tilted). Small rounded hands. Front-facing, arms relaxed at sides. Subtle knowing smile. === FOODOS MASCOT STYLE SYSTEM (see §23.2 Master Style Anchor) ===



03 — FROGGY

Rana tech nerd. Curiosa y con humor.

Cartoon frog character, chibi proportions. Bright lime-green body (#22c55e). Very large round golden eyes. Tiny round glasses (thin black frames). Small white belly patch. Tiny hand scanner device on one wrist. Short rounded arms and legs, wide stance. One eyebrow slightly raised – nerdy expression. Lime-green (#4ade80) highlight on glasses or scanner device. === FOODOS MASCOT STYLE SYSTEM (see §23.2 Master Style Anchor) ===



04 — SAGE

Búho sabio y financiero. Tranquilo y analítico.

Owl character, chibi proportions, compact round body. Deep forest-green and dark brown feathers. Large amber-golden eyes with tiny round tortoiseshell glasses resting on beak. Tiny dark green waistcoat. Calm half-smile expression. Wings folded neatly. Lime-green (#4ade80) accent on glasses lens or waistcoat pocket. Front-facing, dignified but soft pose. === FOODOS MASCOT STYLE SYSTEM (see §23.2 Master Style Anchor) ===



05 — CHIP

Robot IA minimalista. Neutro y eficiente.

Small robot character, chibi proportions with very soft rounded geometric shapes. Matte charcoal body (#1e2a18). Two large circular LED eyes glowing lime-green (#4ade80). Simple antenna on top. Small rounded arms with visible joint connectors. No mouth – all expression through eye shape changes. Tiny chest panel with a small green LED indicator. Front-facing, arms at sides, compact stance. === FOODOS MASCOT STYLE SYSTEM (see §23.2 Master Style Anchor) ===



06 — MUSHI

Seta kawaii plant-based. Soñadora y creativa.

Cute mushroom character, chibi proportions. Pale cream stem/body, pink mushroom cap (#f472b6) with white polka dots. Small rosy cheeks, large dreamy half-closed eyes with sparkle highlights. Tiny leaf or flower sprout on cap. Gentle closed smile. Lime-green (#4ade80) tiny leaf detail or stem stripe. Short stubby arms, front-facing gentle idle pose. === FOODOS MASCOT STYLE SYSTEM (see §23.2 Master Style Anchor) ===



07 — BRUNO

Oso reconfortante de meal prep. Cariñoso y protector.

Chubby bear character, chibi proportions. Warm dark brown fur. Round soft body, large round bear ears. Gentle kind expression – small dark eyes, big rosy cheeks, content smile. Dark green apron over a pale shirt, apron has lime-green (#4ade80) pocket or stitching. Short stubby arms open slightly – inviting pose. Front-facing, feet together, very round and soft shapes throughout. === FOODOS MASCOT STYLE SYSTEM (see §23.2 Master Style Anchor) ===



08 — PICA

Guindilla competitiva. Intensa y retadora.

Chili pepper character, chibi proportions. Bright red elongated body (#ef4444). Female, fierce eyebrows, large intense eyes with flame-like pupils. Small strong fists raised slightly. Sporty lime-green (#4ade80) headband. Determined wide grin showing energy. Tiny leaf stem on head (green). Leaning slightly forward – dynamic attitude pose. Bold black outlines, thick style. === FOODOS MASCOT STYLE SYSTEM (see §23.2 Master Style Anchor) ===



09 — OKTO

Pulpo multitarea. Organizado y divertido.

Octopus character, chibi proportions. Teal-cyan body (#06b6d4). 8 short rounded tentacles – each doing something slightly different (one waves, one points, one holds a tiny phone). Large clever expressive eyes, small content beak-mouth. Lime-green (#4ade80) sucker dots on tentacles. Floating front-facing pose, tentacles spread in a cheerful pattern. === FOODOS MASCOT STYLE SYSTEM (see §23.2 Master Style Anchor) ===



10 — KIRI

Zorro del feed social. Carismático y creativo.

Fox character, chibi proportions. Orange fur (#f97316) with white face and belly markings. Dark ears with white inner color. Confident charming expression – half-smile, slightly raised eyebrow. Slim tail visible behind. Dark turtleneck top with lime-green (#4ade80) collar or cuff detail. Tiny stylish cap tilted. Cool relaxed stance, one arm slightly raised. === FOODOS MASCOT STYLE SYSTEM (see §23.2 Master Style Anchor) ===



11 — VERA

Planta zen del bienestar. Calmada y equilibrada.

Anthropomorphic plant/succulent character, chibi proportions. Soft sage-green rounded body with small leaf-arms (#86efac). Bright lime-green (#4ade80) highlights on leaf tips. Tiny flower crown on head (white petals, yellow center). Peaceful closed-eye smile – zen expression. Hands gently together in front. Very round, soft shapes, no sharp angles. === FOODOS MASCOT STYLE SYSTEM (see §23.2 Master Style Anchor) ===



12 — PINGO

Pingüino del congelador. Metódico y ordenado.

Penguin character, chibi proportions. Classic navy-blue and white coloring. Light blue (#7dd3fc) accent on wings and feet. Small round glasses (square frames). Lime-green (#4ade80) scarf or earmuffs detail. Holding a tiny clipboard or checklist in one wing. Neat meticulous expression – small organized smile. Feet together, upright proud posture. === FOODOS MASCOT STYLE SYSTEM (see §23.2 Master Style Anchor) ===



13 — VOLT

Rayo del ahorro máximo. Hiperactivo y explosivo.

Lightning bolt character, chibi proportions. Bright yellow jagged body (#fde047) shaped like a lightning bolt. Tiny arms and legs sticking out from the bolt shape. Huge wide excited eyes, big open grin. Small electric sparks flying off the edges of the body. Lime-green (#4ade80) inner glow or spark accents. Dynamic leaning-forward pose full of energy. === FOODOS MASCOT STYLE SYSTEM (see §23.2 Master Style Anchor) ===



14 — LEO

León deportivo y proteínico. Fuerte y motivador.

Lion character, chibi proportions. Warm amber-gold mane (#fbbf24), slightly darker fur body. Athletic but still cartoonish – strong arms that are proportionally larger than body. Confident proud half-smile, bright expressive eyes. Dark sports tank top with lime-green (#4ade80) stripe or logo. Relaxed but powerful front-facing stance. === FOODOS MASCOT STYLE SYSTEM (see §23.2 Master Style Anchor) ===



15 — LUNA

Gato nocturno planificador. Misteriosa y tranquila.

Cat character, chibi proportions. Deep indigo-purple fur (#818cf8). Large crescent-shaped eyes with star-shaped pupils – mystical expression. Small pointed ears with inner pink. Fluffy tail visible curling to one side. Dark mini hooded cloak, open at front, with star/moon embroidery. Lime-green (#4ade80) small star detail on cloak or eye catchlight. Calm mysterious front-facing pose, one paw slightly raised. === FOODOS MASCOT STYLE SYSTEM (see §23.2 Master Style Anchor) ===

23.4 Sistema de sprites — cómo crear las animaciones

Cada mascota necesita 9 animaciones (estados). Cada animación es una secuencia de frames que se implementa como un archivo Lottie JSON (la tecnología más eficiente para animaciones vectoriales en web y móvil). A continuación se explica el flujo completo de creación.

Los 9 estados de animación de cada mascota:

Estado	Frames aprox	Tipo	Descripción del movimiento	Trigger en app
idle	12-18	Loop	Respiración suave (cuerpo sube/baja 2-3px). Parpadeo cada 2-3 frames. Estado por defecto siempre activo	Estado por defecto
wave	14-20	Once	Saluda con el brazo/pata. Arco de 90° arriba-abajo. Sonrisa amplia	Al iniciar visita del día
thinking	10-14	Loop	Ojos mirando arriba-derecha. '...' aparece en burbuja. Cabeza precitando	Al precitando / cargando
celebrate	20-28	Once	Salta (cuerpo sube 20px, baja con squash). Brazos arriba. Ojos cerrados feliz macro ok	Ojos cerrados feliz
alert	10-14	Once×2	Se asusta: cuerpo sube 3px, ojos muy abiertos, brazos arriba. Cabeza hacia adelante	Al detectar urgente / alerta
suggest	12-16	Once	Señala con dedo/pata hacia adelante-derecha. Guiño o sonrisa complacida	Recomendación IA nueva
sleep	16-20	Loop	Se sienta o encoge. Ojos cerrados, ZZZ flotando. Respiración muy lenta	Usando >5 min
success_buy	14-18	Once	Thumbs up (o gesto equivalente del personaje). Cara satisfecha	Compra completada
streak	24-32	Once	Baile corto de 2-3 movimientos. Estrellas o destellos alrededor	7+ días sin desperdiciar

23.5 Flujo completo de creación de sprites paso a paso

PASO 1

Generar la imagen base del personaje

Usa el prompt individual del §23.3 + el Master Style Anchor del §23.2 en Google AI Studio (Imagen 3) o Leonardo AI. Genera 4 variaciones y elige la mejor. Descarga en PNG 512×512 con fondo transparente. Si el fondo no es transparente, pásala por remove.bg (50 usos/mes gratis). Esta imagen base es el 'canon' del personaje — todas las animaciones deben parecerse a ella.

Prompt extra para asegurar consistencia en poses posteriores:

"Same character as reference image [adjuntar PNG], but now in [POSE]:

[describir la pose]. Keep EXACTLY the same art style, colors, proportions,

and outline thickness. Only change the pose and expression.

=== FOODOS MASCOT STYLE SYSTEM ===

PASO 2

Generar las poses clave de cada animación

Para cada uno de los 9 estados, genera la pose clave (el frame más expresivo). Ejemplo: para 'celebrate' la pose clave es el momento en que el personaje está en el aire con los brazos arriba. Para 'sleep' es el personaje sentado con ojos cerrados. Adjunta siempre la imagen base como referencia para mantener consistencia.

Estructura de poses clave a generar por personaje (9 imágenes PNG):

- idle_base.png → pose neutral, ojos abiertos, leve sonrisa

- wave_peak.png → brazo en alto, gran sonrisa
- thinking.png → cabeza ladeada, ojos arriba, boca neutral
- celebrate.png → en el aire, brazos arriba, ojos cerrados (feliz)
- alert.png → ojos MUY abiertos, brazos arriba, boca pequeña "o"
- suggest.png → dedo señalando hacia delante, guiño
- sleep.png → ojos cerrados, sentado/encogido, ZZZ
- success.png → thumbs up (o equivalente), cara satisfecha
- streak.png → pose de baile exagerada, destellos alrededor

PASO 3 Importar en Rive y animar

Rive (rive.app) es la herramienta de animación más potente para esto y es gratis. Importa la imagen base como PNG en Rive. Usa la herramienta Bones para crear un esqueleto simple (5-7 huesos: cabeza, torso, brazo izq, brazo der, piernas). Crea una State Machine con los 9 estados. Para cada estado, anima el esqueleto entre la pose neutral y la pose clave. Rive exporta directamente como .riv (nativo) o como Lottie JSON.

Estructura de bones recomendada para personajes chibi FoodOS:

Root

- body ← todo el cuerpo, para el bounce del idle
- ■■■ head ← movimiento de cabeza (thinking, wave)
- ■ ■■■ eyes ← parpadeo (idle, sleep)
- ■ ■■■ mouth ← expresiones
- ■■■ arm_L ← brazo izquierdo
- ■■■ arm_R ← brazo derecho (wave, celebrate, suggest)
- feet ← para el jump de celebrate

PASO 4 Alternativa sin Rive — CSS sprite sheet

Si no quieres usar Rive/Lottie, la alternativa más sencilla es un sprite sheet PNG: una imagen con todos los frames en fila. La animación se hace con CSS animation cambiando background-position. Es menos flexible pero más fácil de crear. Genera cada frame individualmente con IA (pide la misma pose en distintas posiciones) y junta los frames en Photopea (gratis) o ezgif.com.

Especificación del sprite sheet para FoodOS:

- Tamaño por frame: 128×128 px
- Frames por fila: máximo 8
- Nombre: [personaje]_[estado].png ej: zana_idle.png
- Formato: PNG con transparencia (32-bit)
- Resolución de display: 64×64 px (retina: 128×128)

Ejemplo CSS para animar sprite sheet idle (12 frames):

```
.mascot-idle {  
  
  width: 64px; height: 64px;  
  
  background: url('/mascots/zana_idle.png') 0 0;  
  
  background-size: 512px 128px; /* 8 frames × 64px */  
  
  animation: zana-idle 1.2s steps(8) infinite;  
  
}  
  
@keyframes zana-idle {  
  
  to { background-position: -512px 0; }  
  
}
```

PASO 5 Exportar e integrar en FoodOS

Desde Rive exporta como .riv para máximo rendimiento o como Lottie JSON para compatibilidad. Coloca los archivos en /public/mascots/[nombre]/. El componente MascotWidget.tsx del §23.7 los carga dinámicamente — solo carga el personaje activo del usuario, no los 15 a la vez.

Checklist antes de integrar cada animación:

- ✓ El personaje se ve correctamente en fondo oscuro (#070a05)
- ✓ El PNG tiene fondo transparente (no blanco)
- ✓ El tamaño del archivo Lottie es < 50 KB por animación
- ✓ El loop de idle es seamless (el último frame conecta con el primero)
- ✓ Las animaciones "once" no dejan el personaje en pose rara al terminar
- ✓ Probado en móvil (tamaño 64×64 se ve bien)
- ✓ Accesibilidad: animación se desactiva con prefers-reduced-motion

23.6 Herramientas necesarias — todas gratuitas

Herramienta	URL	Para qué	Gratis
Google AI Studio	aistudio.google.com	Generar imágenes base (Imagen 3)	Con AI Plus
Leonardo AI	app.leonardo.ai	Variaciones y consistencia de estilo	150 tokens/día
Adobe Firefly	firefly.adobe.com	Alternativa para generar personajes	25 créditos/mes
Rive	rive.app	Animar el personaje, exportar Lottie	100% gratis
LottieFiles Creator	creator.lottiefiles.com	Editor online de Lottie (más simple)	100% gratis
LottieFiles Library	lottiefiles.com	Animaciones base adaptables como referencia	Gratis (con cuenta)
Photopea	photopea.com	Editar PNG, montar sprite sheets	100% gratis
remove.bg	remove.bg	Eliminar fondo de PNG generados	50/mes gratis
ezgif.com	ezgif.com/sprite-cutter	Cortar y montar sprite sheets	100% gratis
FalSprite	github.com/lovisdotio/falsprite	Generar sprite sheet animado con prompt	fal.ai API key
Piskel	piskelapp.com	Editor online de sprites (pixel art)	100% gratis

23.7 Mantener consistencia visual entre los 15 personajes

El mayor riesgo al crear 15 personajes con IA es la inconsistencia: que cada uno parezca de una app diferente. Estas son las reglas para evitarlo:

- **Siempre adjuntar referencia:** al generar variaciones o poses, adjunta siempre la imagen base aprobada del personaje como reference image en el prompt. Nunca generes de cero.
- **Mismo generador para el mismo personaje:** si empezaste a Zana en Leonardo AI, continúa todas sus poses en Leonardo AI. No mezcles generadores para el mismo personaje.
- **El Master Style Anchor no cambia nunca:** cópialo literalmente en cada prompt. No lo parafrasees ni lo acortes.
- **Documenta el seed/modelo usado:** guarda el número de seed o el nombre exacto del modelo que generó la imagen base aprobada. Úsalo para todas las variaciones.
- **Genera todos los estados antes de pasar al siguiente personaje:** no crees Zana idle, luego Basil idle, etc. Termina los 9 estados de Zana, luego los 9 de Basil.
- **Test visual en grid:** antes de implementar, pon los 15 personajes en una cuadrícula 5×3 y visualízalos juntos. Si alguno desentona visualmente, regenerarlo.
- **El outline negro es innegociable:** si un generador produce un personaje sin outline, recházalo o usa un post-proceso para añadirlo (Photopea: Filter → Stylize → Find Edges adaptado).

23.8 Pantalla de selección de mascota en el onboarding

La elección de mascota es el último paso del onboarding (paso 4 de 4). El usuario puede cambiar su mascota en cualquier momento desde Ajustes → Mi mascota sin perder ningún dato.

- Grid de 3 columnas con scroll vertical. 15 personajes = 5 filas.
- Cada celda: animación Lottie 'idle' en loop + nombre + emoji. Tamaño 80×80px.
- Al tocar: borde del color del personaje activo, animación 'wave' se reproduce una vez.

- Bajo el grid: nombre del personaje seleccionado en grande + tagline en cursiva.
- Botón 'Este es mi compañero' en verde lima fijo en la parte inferior.
- No hay personaje 'mejor' que otro — se comunica explícitamente: 'Elige el que más te guste. Puedes cambiarlo cuando quieras.'
- La selección se guarda en Supabase en `user_profiles.mascot_id` (tipo TEXT, default 'zana').

23.9 Implementación técnica — código completo

```
// types/mascot.ts

// types/mascot.ts

export type MascotId =

  | 'zana' | 'basil' | 'froggy' | 'sage' | 'chip'

  | 'mushi' | 'bruno' | 'pica' | 'okto' | 'kiri'

  | 'vera' | 'pingo' | 'volt' | 'leo' | 'luna';

export type MascotState =

  | 'idle' | 'wave' | 'thinking' | 'celebrate'

  | 'alert' | 'suggest' | 'sleep' | 'success_buy' | 'streak';

export interface MascotMeta {

  id: MascotId; name: string; emoji: string; color: string; tagline: string;

}

export const MASCOTS: MascotMeta[] = [

  { id:'zana', name:'Zana', emoji:'■', color:'#fb923c', tagline:'Energética y motivadora' },
  { id:'basil', name:'Basil', emoji:'■■■■', color:'#a3e635', tagline:'Sereno y experto' },
  { id:'froggy',name:'Froggy',emoji:'■', color:'#22c55e', tagline:'Curioso y con humor' },
  { id:'sage', name:'Sage', emoji:'■', color:'#c084fc', tagline:'Tranquilo y analítico' },
  { id:'chip', name:'Chip', emoji:'■', color:'#60a5fa', tagline:'Neutro y eficiente' },
  { id:'mushi', name:'Mushi', emoji:'■', color:'#f472b6', tagline:'Soñadora y creativa' },
  { id:'bruno', name:'Bruno', emoji:'■', color:'#a78bfa', tagline:'Cariñoso y protector' },
  { id:'pica', name:'Pica', emoji:'■■■', color:'#ef4444', tagline:'Intensa y retadora' },
  { id:'okto', name:'Okto', emoji:'■', color:'#06b6d4', tagline:'Organizado y multitarea' },
  { id:'kiri', name:'Kiri', emoji:'■', color:'#f97316', tagline:'Carismático y creativo' },
  { id:'vera', name:'Vera', emoji:'■', color:'#86efac', tagline:'Calmada y equilibrada' },
  { id:'pingo', name:'Pingo', emoji:'■', color:'#7dd3fc', tagline:'Metódico y ordenado' },
  { id:'volt', name:'Volt', emoji:'■', color:'#fde047', tagline:'Hiperactivo y explosivo' },
  { id:'leo', name:'Leo', emoji:'■', color:'#fbbf24', tagline:'Fuerte y motivador' },
```

```
{ id:'luna', name:'Luna', emoji:'🌙', color:'#818cf8', tagline:'Misteriosa y tranquila' },
];
```

```
// stores/mascot-store.ts
```

```
// stores/mascot-store.ts – Zustand store global persistido
```

```
import { create } from 'zustand';
```

```
import { persist } from 'zustand/middleware';
```

```
import type { MascotId, MascotState } from '@types/mascot';
```

```
interface MascotStore {
```

```
mascotId: MascotId;
```

```
state: MascotState;
```

```
message: string | null;
```

```
setMascot: (id: MascotId) => void;
```

```
trigger: (state: MascotState, msg?: string) => void;
```

}

```
const LOOP_STATES: MascotState[] = ['idle', 'thinking', 'sleep'];
```

```
export const useMascotStore = create<MascotStore>()
```

```
persist((set) => ({
```

```
mascotId: 'zana',
```

```
state: 'idle',
```

```
message: null,
```

```
setMascot: (id) => set({ mascotId: id, state: 'idle', message: null } ),
```

```
trigger: (state, msg) => {
```

```
set({ state, message: msg ?? null });
```

```
if (!LOOP_STATES.includes(state)) {
```

```
// Auto-vuelve a idle tras la duración de la animación
```

```
setTimeout(() => set({ state: 'idle', message: null }), 2800);
```

}

 $\}$

```
)), { name: 'foodos-mascot-v1' })
```

 $\cdot$ [illegible]

```
// Cuando el usuario completa macros del día:
```

```
// useMascotStore.getState().trigger('celebrate', '¡Lo lograste hoy!');

// Cuando caduca un alimento:

// useMascotStore.getState().trigger('alert', 'La leche caduca mañana.');
```

```
// Durante carga de IA:

// useMascotStore.getState().trigger('thinking');

// → luego al terminar:

// useMascotStore.getState().trigger('suggest', '3 recetas encontradas.');
```

```
// components/MascotWidget.tsx

// components/MascotWidget.tsx – Widget flotante completo

'use client';

import { use, useEffect, useState } from 'react';

import dynamic from 'next/dynamic';

import { useMascotStore } from '@stores/mascot-store';

import { MASCOTS } from '@types/mascot';

// Lottie se carga solo en el cliente (no SSR)

const Lottie = dynamic(() => import('lottie-react'), { ssr: false });

export function MascotWidget() {

  const { mascotId, state, message } = useMascotStore();

  const [animData, setAnimData] = useState<object | null>(null);

  const [bubble, setBubble] = useState(false);

  const meta = MASCOTS.find(m => m.id === mascotId!);

  // Cargar JSON de la animación actual (lazy – solo cuando cambia)

  useEffect(() => {

    let cancelled = false;

    import(`@/public/mascots/${mascotId}/${state}.json`)

      .then(m => { if (!cancelled) setAnimData(m.default); })

      .catch(() =>

        import(`@/public/mascots/${mascotId}/idle.json`)

          .then(m => { if (!cancelled) setAnimData(m.default); })

      );

    return () => { cancelled = true; };

  }, [mascotId, state]);
```



```

// Bubble aparece con mensaje, desaparece a los 4s

useEffect(() => {

  if (!message) return;

  setBubble(true);

  const t = setTimeout(() => setBubble(false), 4000);

  return () => clearTimeout(t);

}, [message]);

return (

  <div className="fixed bottom-6 right-6 z-50 flex flex-col items-end gap-2

    pointer-events-none">

    {bubble && (

      <div className="max-w-52 rounded-2xl rounded-br-sm px-3 py-2

        text-xs font-medium shadow-xl pointer-events-auto

        animate-in slide-in-from-bottom-2 duration-200"

        style={{ background: meta.color + '20', border: `1px solid ${meta.color}50`,

          color: meta.color }}>

        {message}

      </div>

    )}

    <div className="w-16 h-16 pointer-events-auto cursor-pointer

      hover:scale-110 active:scale-95 transition-transform duration-150"

      onClick={() => useMascotStore.getState().trigger('wave')}

      title={meta.name}>

      {animData

        ? <Lottie animationData={animData}

          loop={['idle', 'thinking', 'sleep'].includes(state)}

          autoplay style={{ width: 64, height: 64 }} />

        : <span className="text-4xl">{meta.emoji}</span>

      }

    </div>

  </div>

);

}

```

Estructura de archivos:

// Estructura de archivos de los 150 assets de mascotas

```
public/mascots/
```

```
■■■ zana/
```

```
■   ■■■ idle.json           (< 30KB)
■   ■■■ wave.json          (< 25KB)
■   ■■■ thinking.json      (< 20KB)
■   ■■■ celebrate.json     (< 40KB)
■   ■■■ alert.json         (< 20KB)
■   ■■■ suggest.json       (< 20KB)
■   ■■■ sleep.json         (< 25KB)
■   ■■■ success_buy.json   (< 25KB)
■   ■■■ streak.json        (< 45KB)
■   ■■■ preview.png        (128×128, PNG)
```

```
■■■ basil/ [misma estructura]
```

```
■■■ froggy/ [misma estructura]
```

```
...
```

```
■■■ luna/ [misma estructura]
```

Total: 15 × 10 archivos = 150 archivos

Peso máximo estimado: 15 × (9×30KB avg + 10KB PNG) = ~4.2 MB

Solo se cargan los archivos del personaje activo del usuario.

ORDEN DE IMPLEMENTACIÓN RECOMENDADO

EMPEZAR POR ZANA Y CHIP: Zana porque es el personaje más reconocible y el que más aparecerá en screenshots y marketing. Chip porque sus formas geométricas simples son las más fáciles de animar en Rive. Con esos dos validados, el proceso para los 13 restantes es repetible y rápido.